



LLM Serving and RAG

Youngjae Kim (Ph.D.)

Sogang University

Linux 운영체제 및 응용 (GITF315-01)



Transformer & LLM Inference

Transformer: Encoder-Decoder Model

Transformer Overview

- Proposed in “**Attention is All You Need**” (Google Brain and U. of Toronto, Vaswani et al., 2017)
- Designed for Neural Machine Translation (NMT)
- Uses only Self-Attention, no recurrence (RNN/순환신경망) or convolution (CNN/합성곱신경망)
- Became the foundation of modern NLP

Two Main Components

- Encoder**
 - Encodes the source sentence
 - Builds contextual meaning representations via Self-Attention
- Decoder**
 - Generates the target sentence one token at a time
 - Uses **Self-Attention** (on generated tokens so far)
 - Uses Cross-Attention to incorporate Encoder outputs

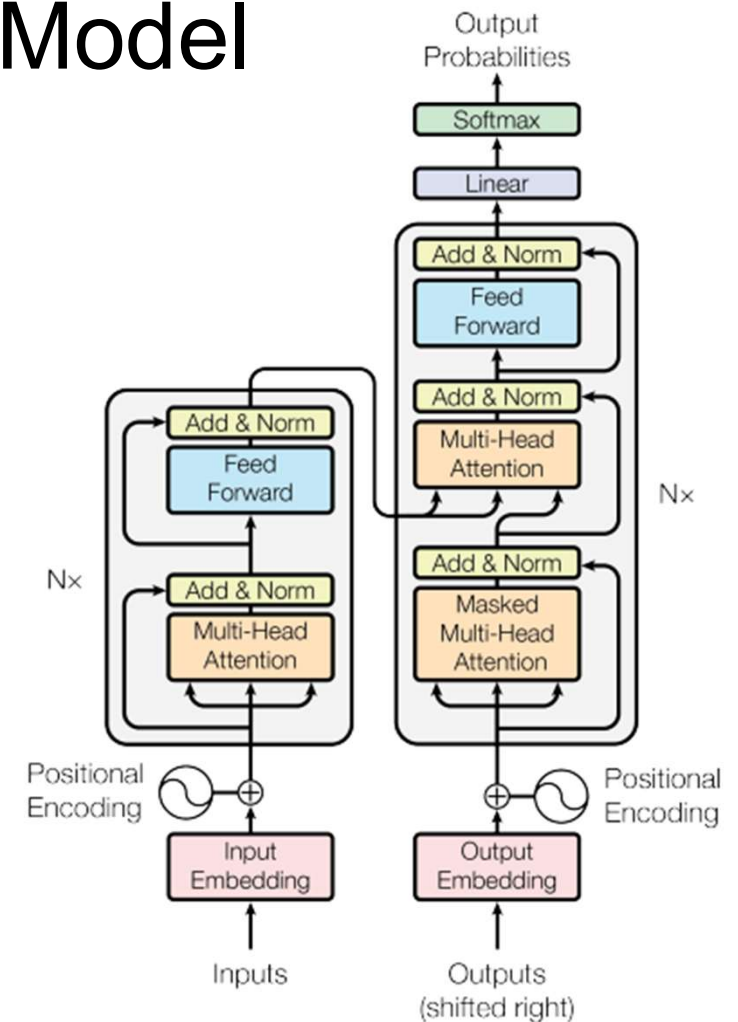


Figure 1: The Transformer - model architecture.

[1] Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

Transformer: Overall Architecture

Encoder / Decoder 사용하는 아키텍처는 동일함.

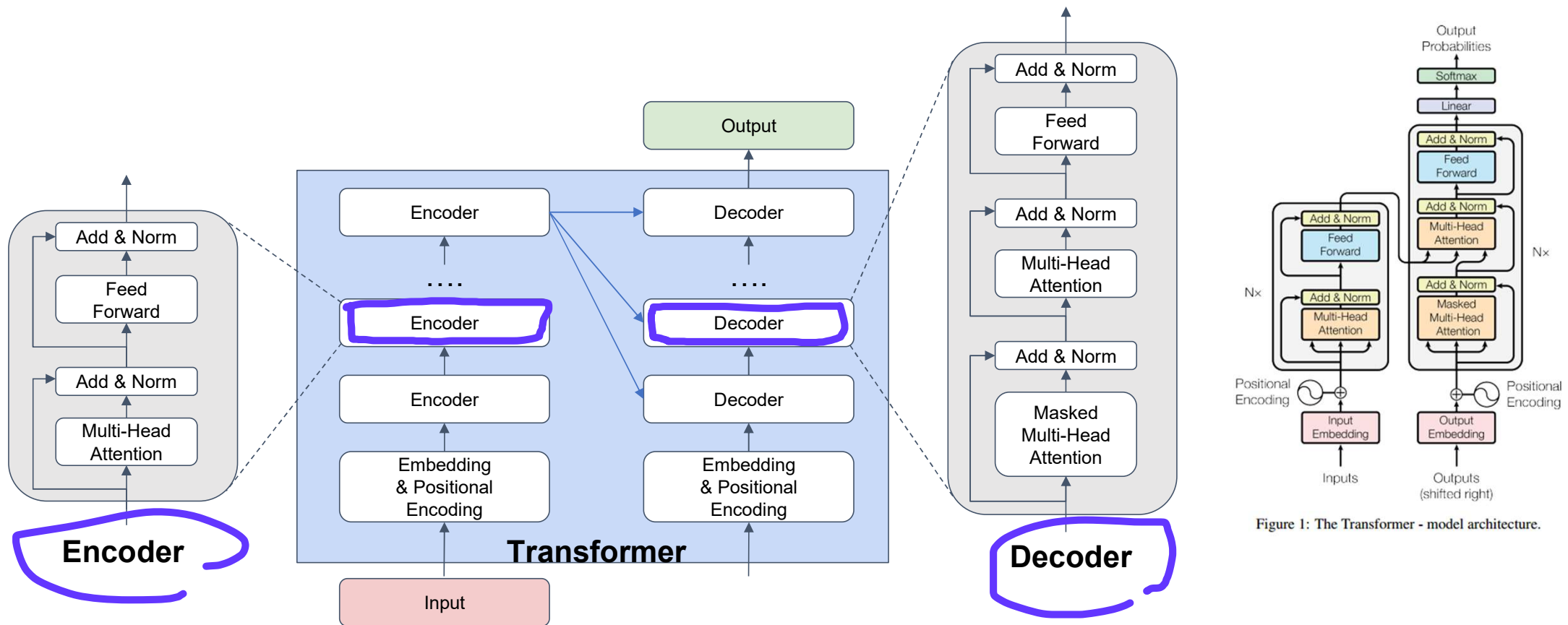
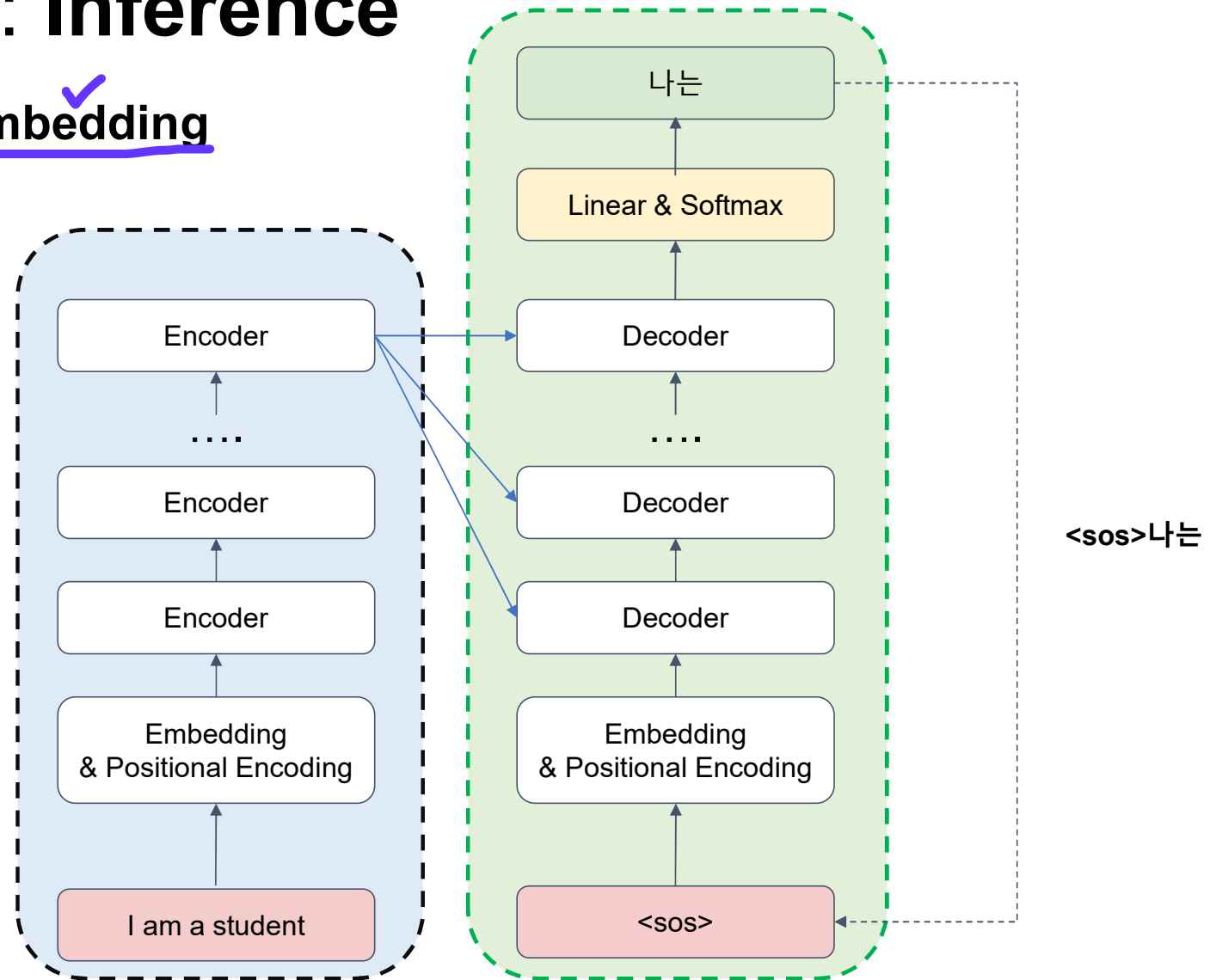


Figure 1: The Transformer - model architecture.

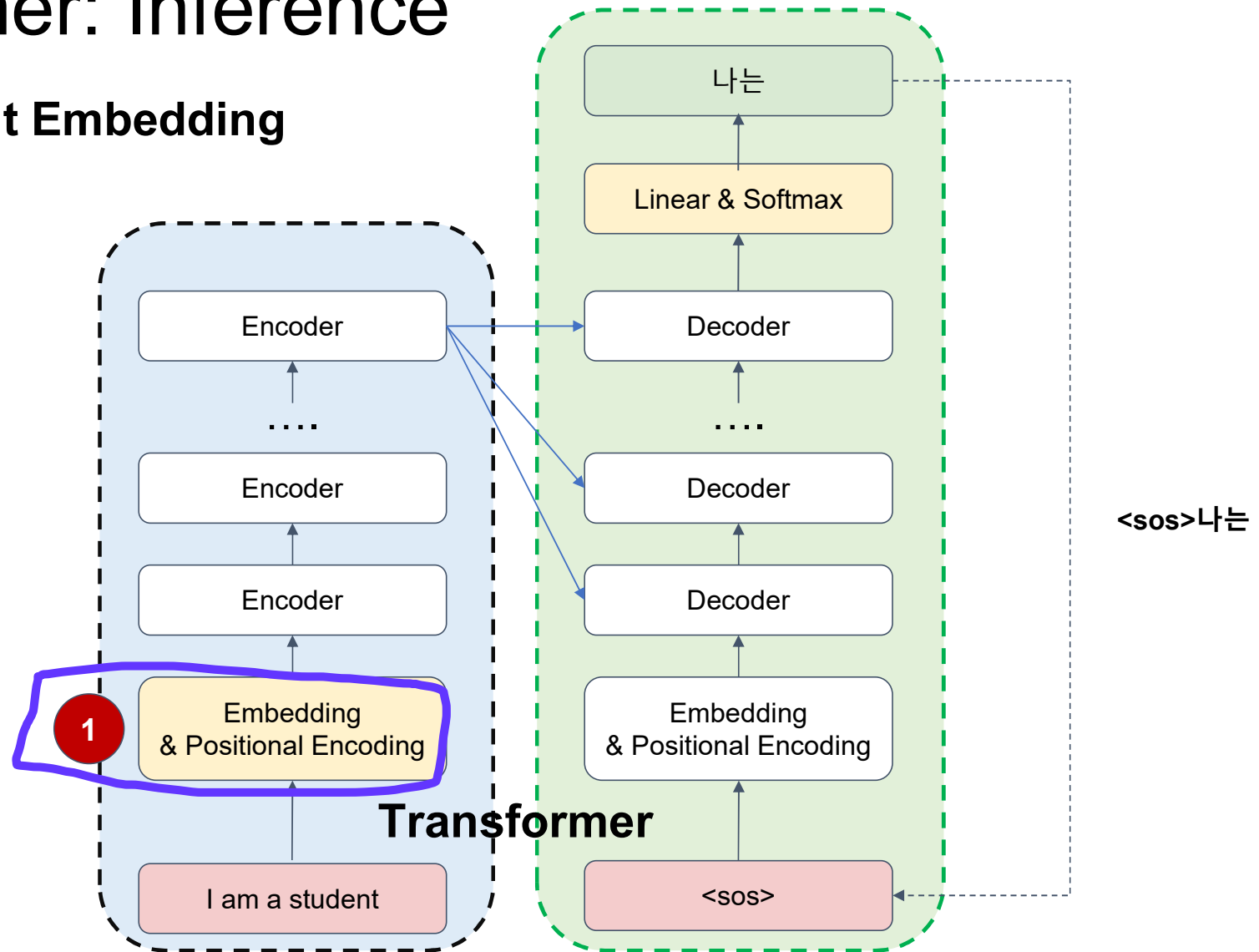
Transformer: Inference

1. Encoder- Input Embedding



Transformer: Inference

1. Encoder- Input Embedding



Transformer: Inference

1. Encoder- Input Embedding

1.1. Tokenization

['I', 'am', 'a', 'student']

1.2. Embedding

Ex) 768 ~ 12,288 dimensions
in GPT-3 series

embedding

"I"	1.1	0.2	...	0.3
"am"	0.1	2.1		2.3
"a"	0.5	0.1	...	1.1
"student"	0.2	1.5		0.4

Embedding dimension
($d_{\text{model}} = 512$)

1.3. Positional Encoding

Input
Embedding

1.1	0.2	...	0.3
0.1	2.1	...	2.3
0.5	0.1	...	1.1
0.2	1.5	...	0.4

+

Positional
Encoding

gray	white	white	white
gray	gray	white	white
white	gray	gray	white
white	white	gray	gray

=

Embedded
Token
Matrix

orange	orange	orange	orange
orange	orange	orange	orange
orange	orange	orange	orange
orange	orange	orange	orange

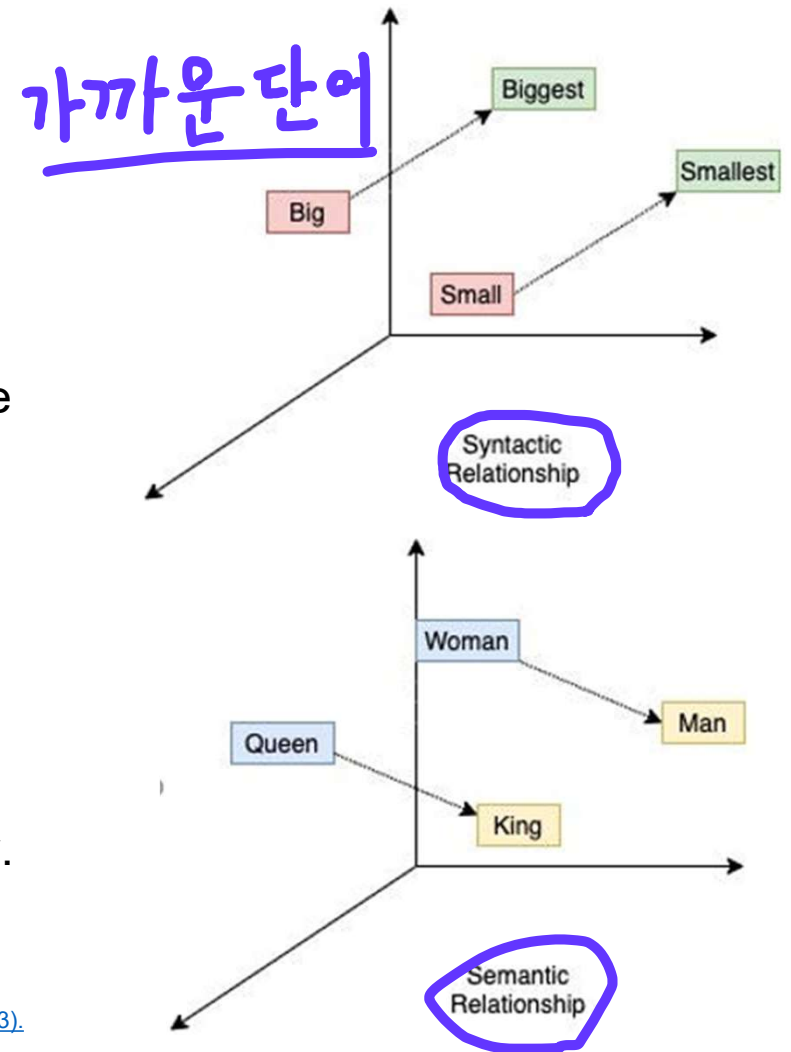
Transformer: Inference

1. Encoder- Input Embedding

↳ Embedding: Maps discrete tokens into continuous vectors.

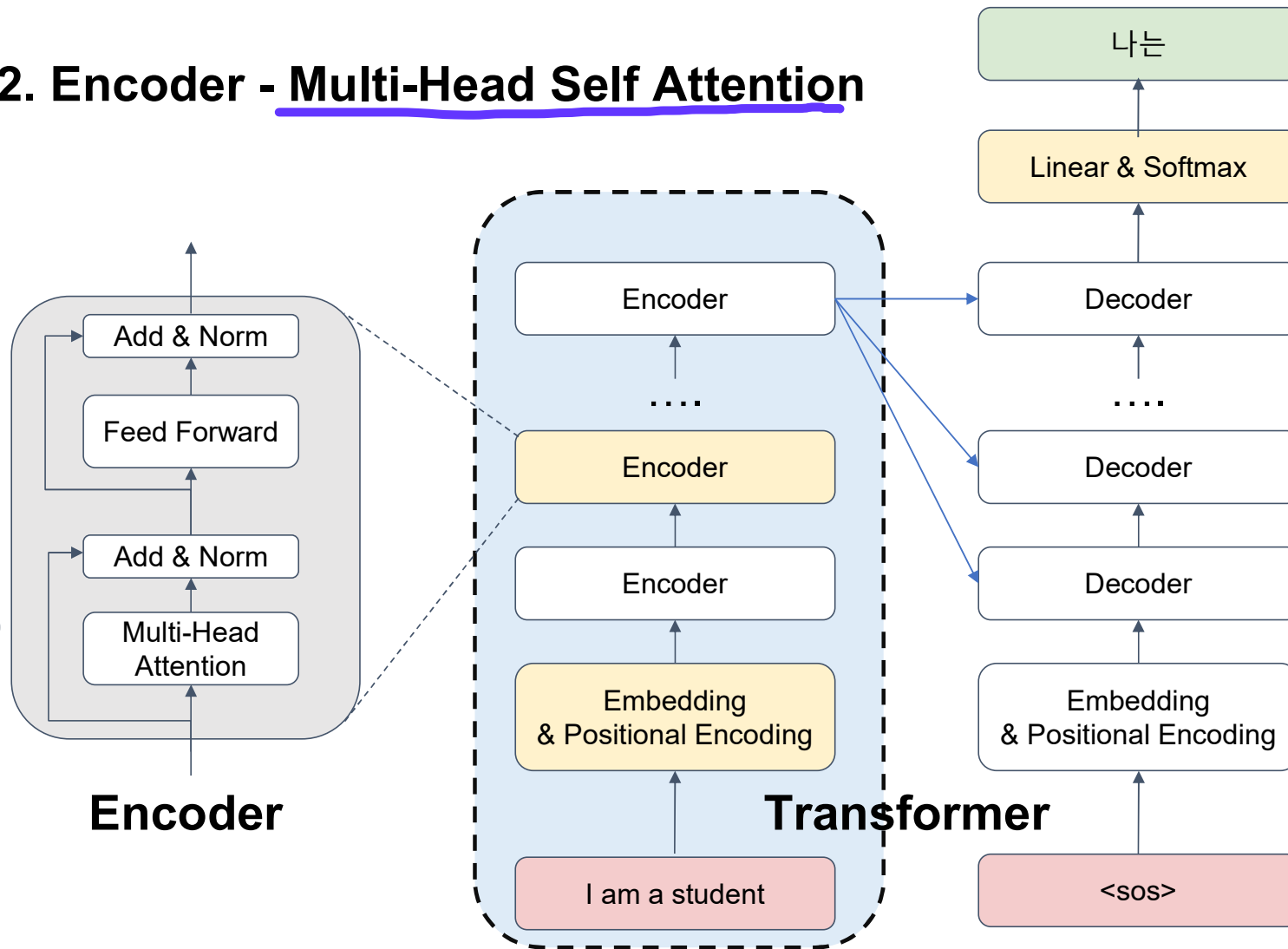
- Word-level: Word2Vec (Mikolov et al., 2013)
 - Assigns a **fixed vector** to each word to capture **word** similarity.
- Sentence/document-level: OpenAI Embeddings, Sentence-BERT
 - **Dynamically** encodes **entire sentences** with context to capture precise semantic similarity.
 - Ex) river bank (강둑) vs. financial bank (은행)

[2] Mikolov, Tomas, et al. "Efficient estimation of word representations in vector space." *arXiv preprint arXiv:1301.3781* (2013).



Transformer: Inference

2. Encoder - Multi-Head Self Attention



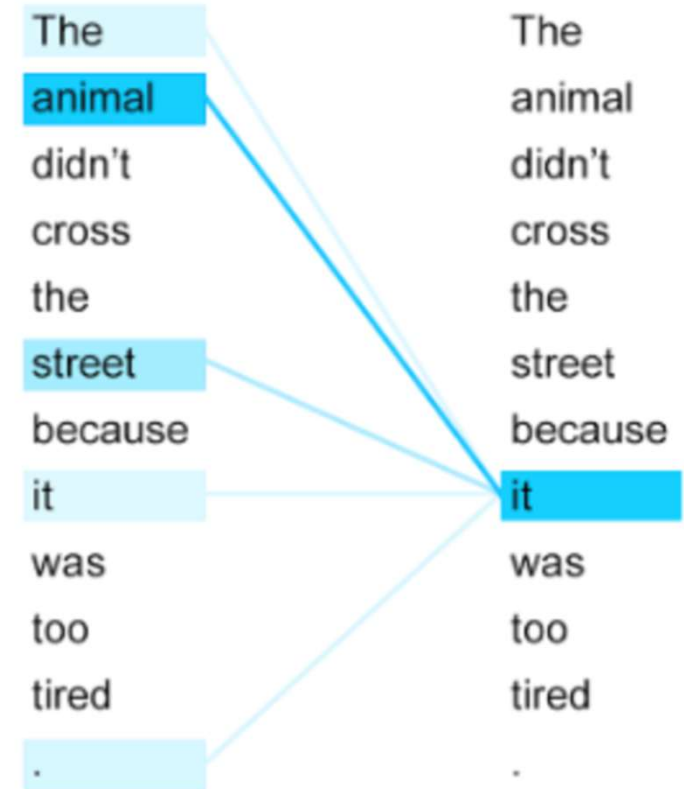
Transformer: Inference

2. Encoder - Multi-Head Self Attention

🔗 Calculate relevance between tokens.

🔗 “The animal didn’t cross the street because it was too tired.”

- “it” → “animal” or “street”?
- **Attention Scores** → “it” is more relevant to “animal” than “street”!



Transformer: Inference

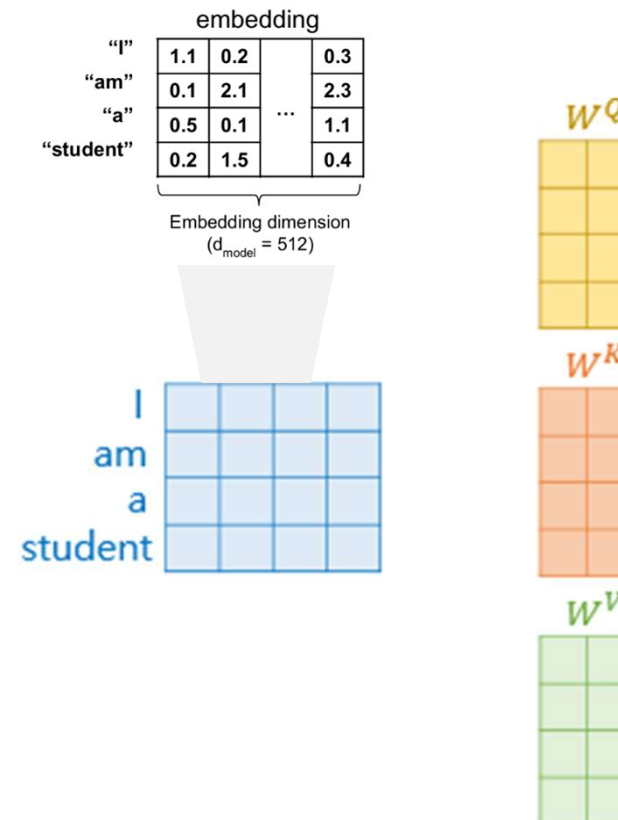
2. Encoder - Multi-Head Self Attention

🔗 Self-Attention Basics

- Each token is transformed into Query (Q), Key (K), and Value (V)
- Q, K, V capture different roles in attention:
 - Query** → what information is sought
 - Key** → the “address” or feature of a token
 - Value** → the actual content/information

🔗 Weight Matrices

- W^Q , W^K , W^V : trainable weight matrices
- Map input embeddings into Q, K, V spaces

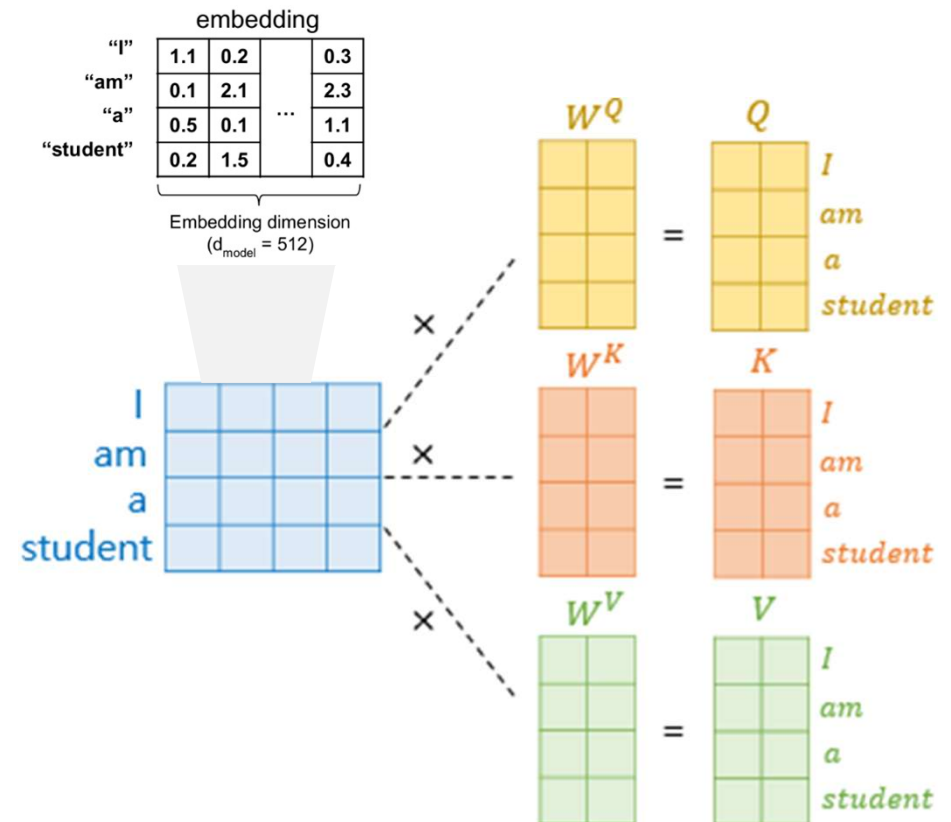


Transformer: Inference

2. Encoder - Multi-Head Self Attention

Self-Attention in Encoder

- In Encoder, Q, K, and V all come from the same input sequence
- Every token attends to all other tokens
- Captures contextual relationships across the sequence

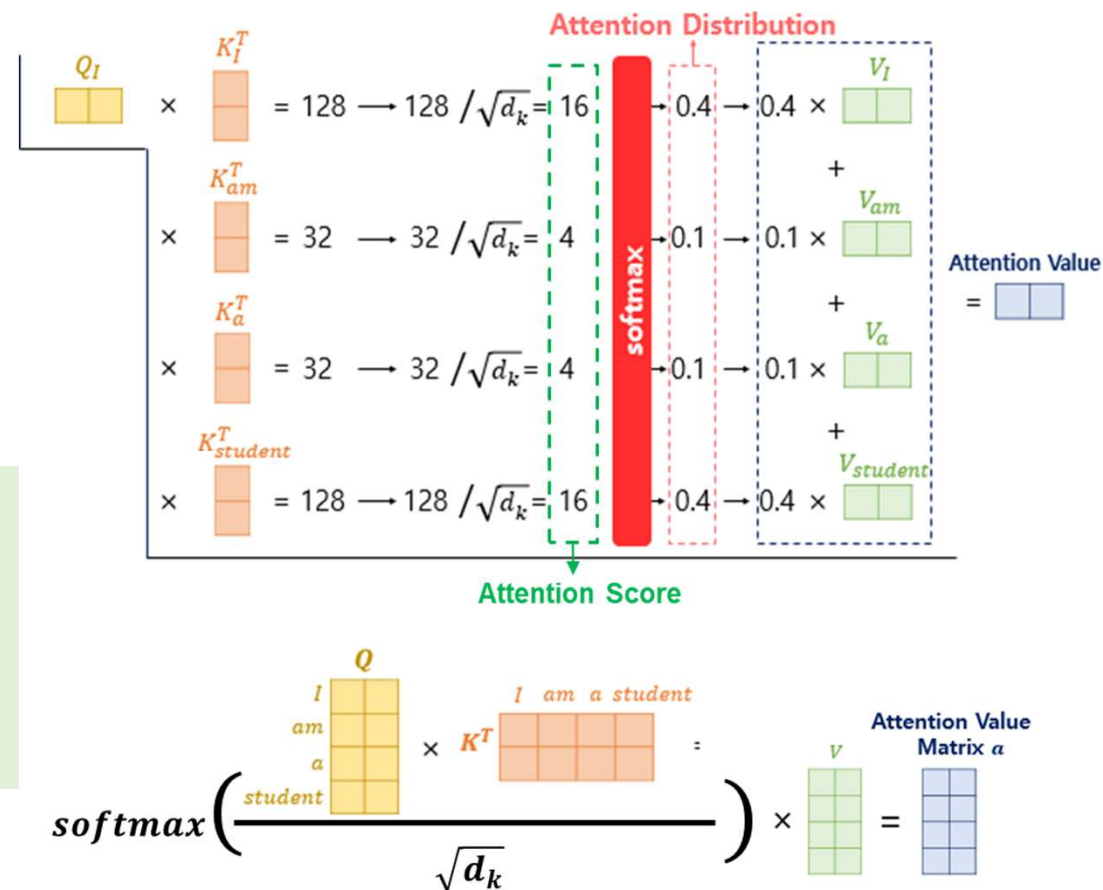
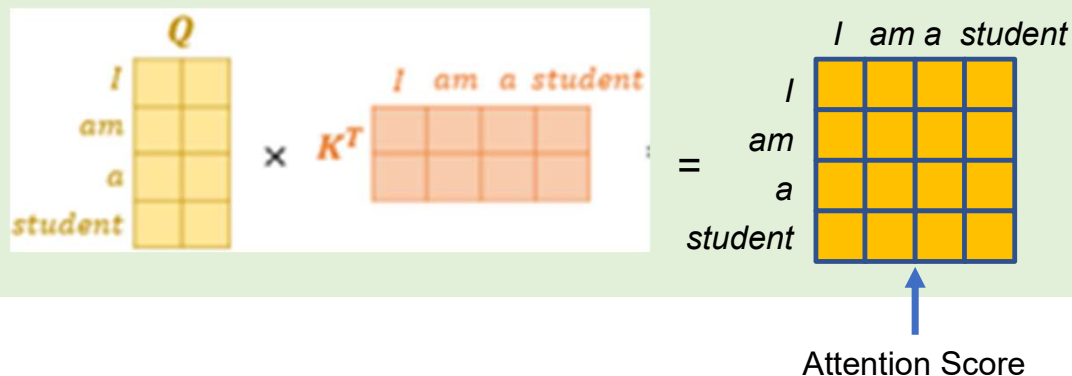


Transformer: Inference

2. Encoder - Multi-Head Self Attention

🔗 Scaled Dot Product Attention (in 1 head)

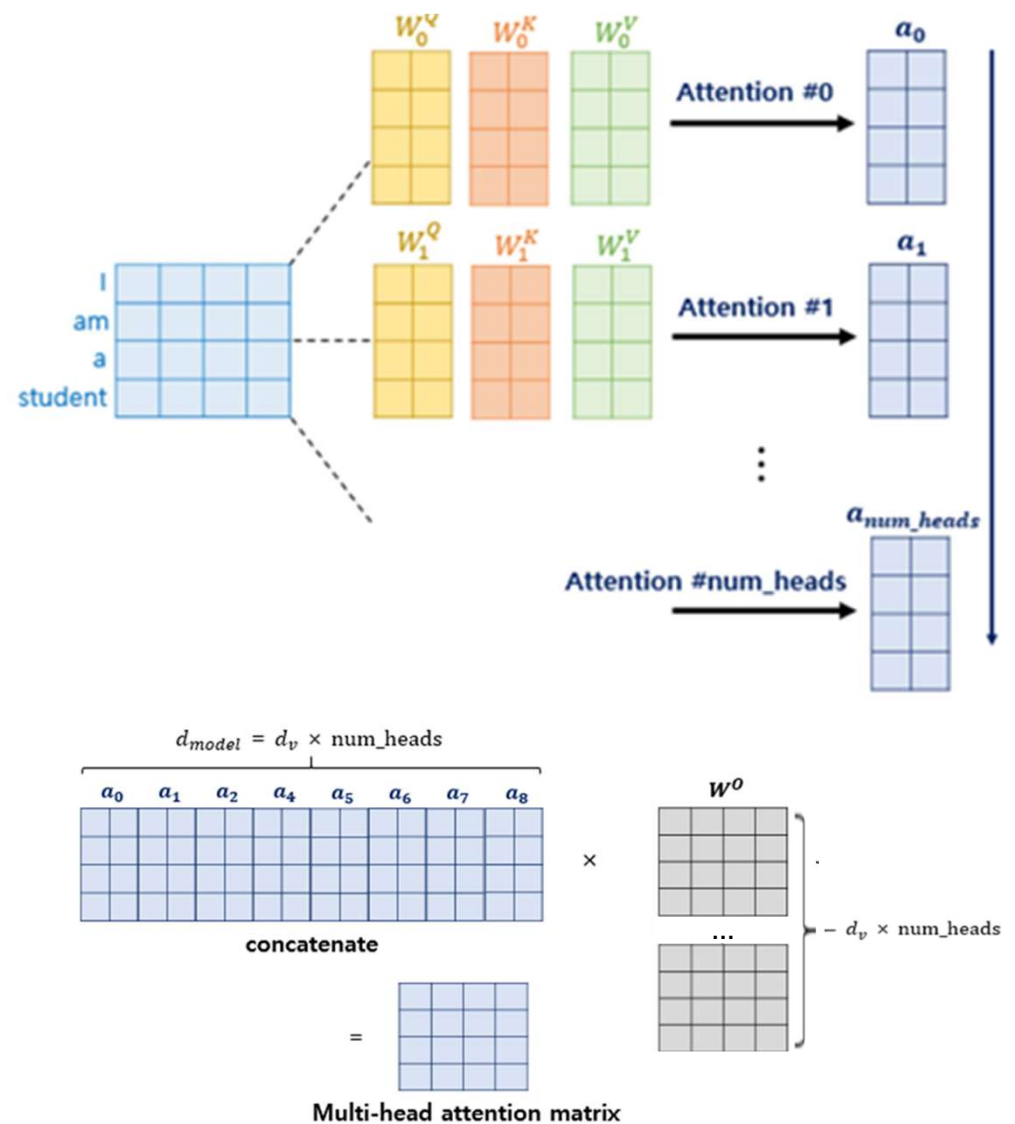
- **Attention score**
 - Weight of Key with respect to Query
- **Attention value**
 - Weighted sum of Value
 - **Contextualized token representation**



Transformer: Inference

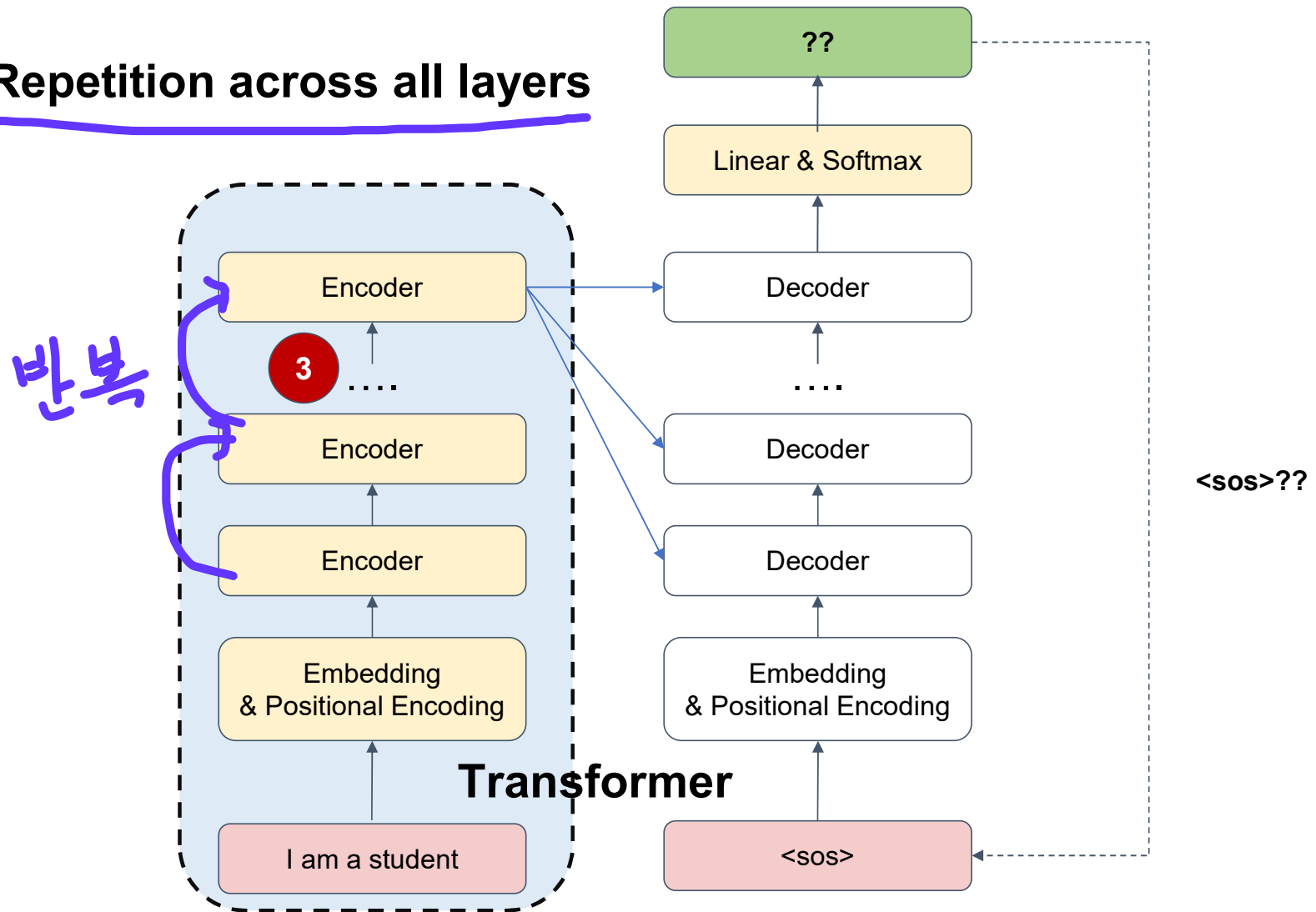
2. Encoder - Multi-Head Self Attention

- Multiple sets of Q, K, V projections (heads)
 - Each head learns different types of relationships
 - Syntax, semantics, long-range dependencies
 - Outputs are concatenated and combined → richer representations
-
- W^Q, W^K, W^V all differ for each Attention Heads.
 - Additional Weight Matrix W^O
 - Concatenate each attention result then multiply by W^O to get final attention matrix.
 - Project the **concatenated multi-head outputs back to the same dimension as the input.**



Transformer: Inference

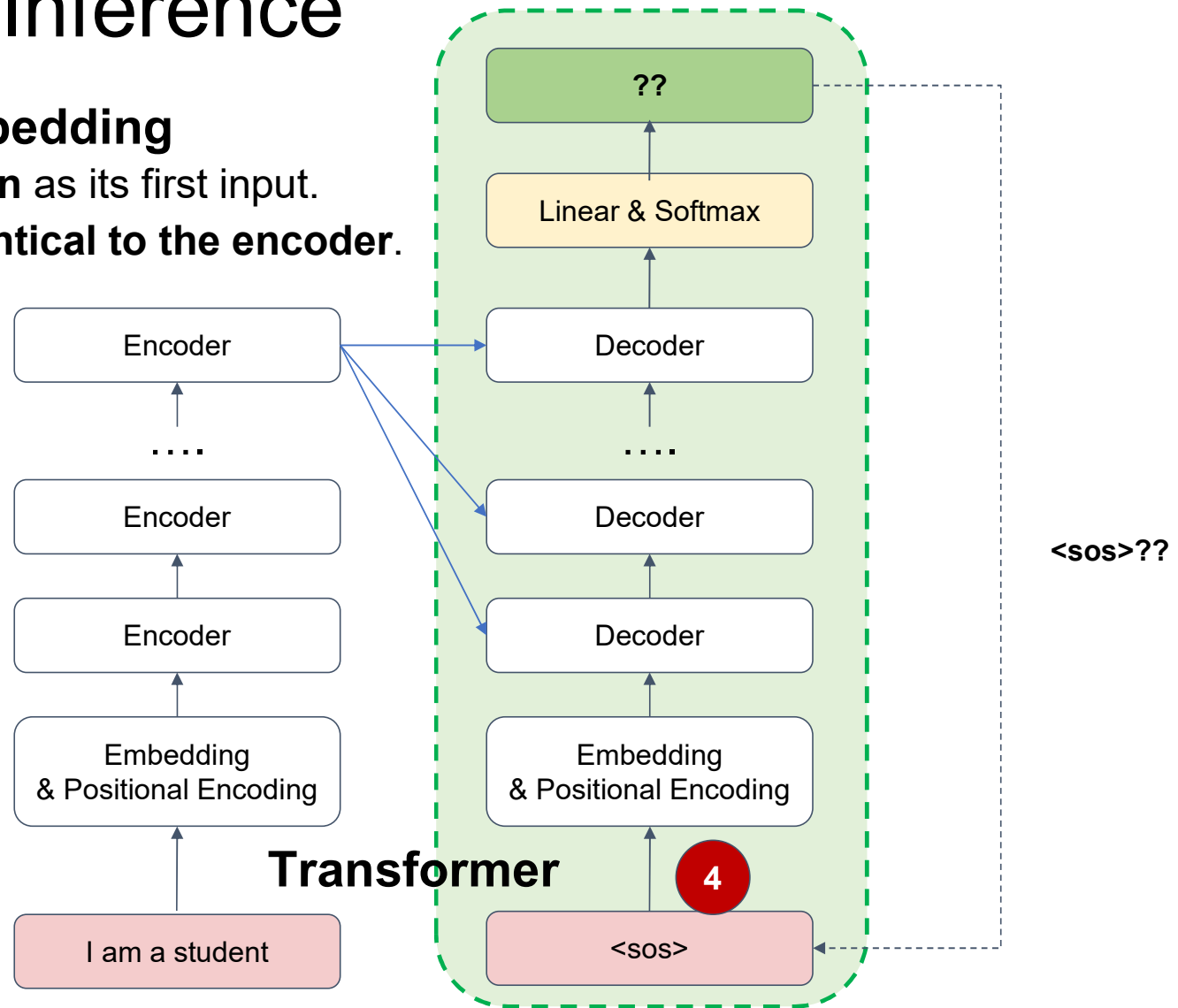
3. Encoder- Repetition across all layers



Transformer: Inference

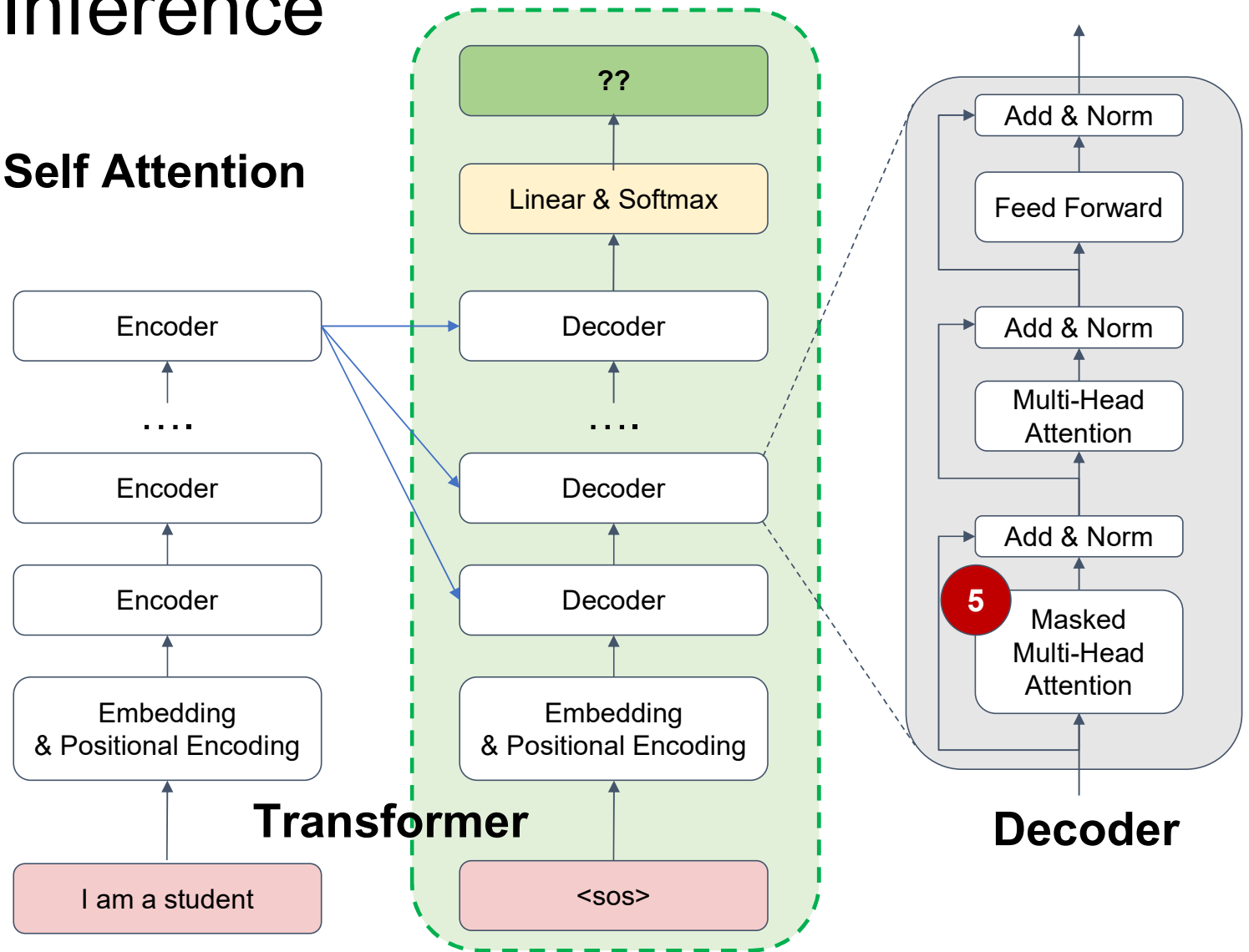
4. Decoder- Input Embedding

- Special **<sos>** token as its first input.
- Other steps are **identical to the encoder**.



Transformer: Inference

5. Decoder – Masked Multi-Head Self Attention

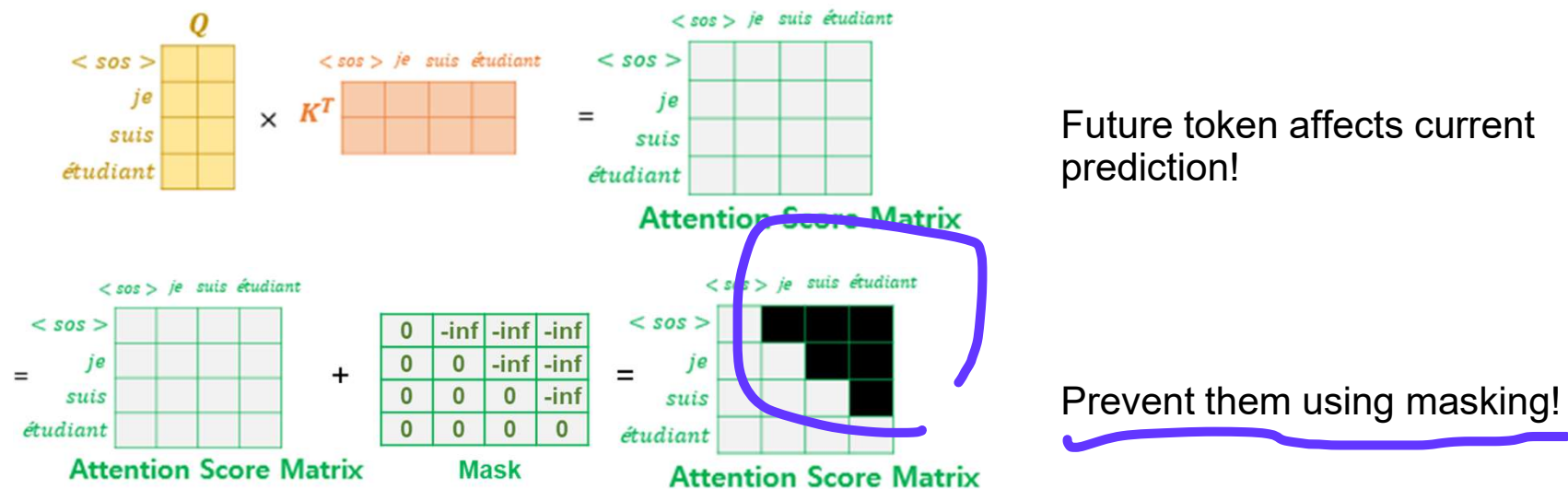


Transformer: Inference

5. Decoder - Masked Multi-Head Self Attention

⌘ Training phase

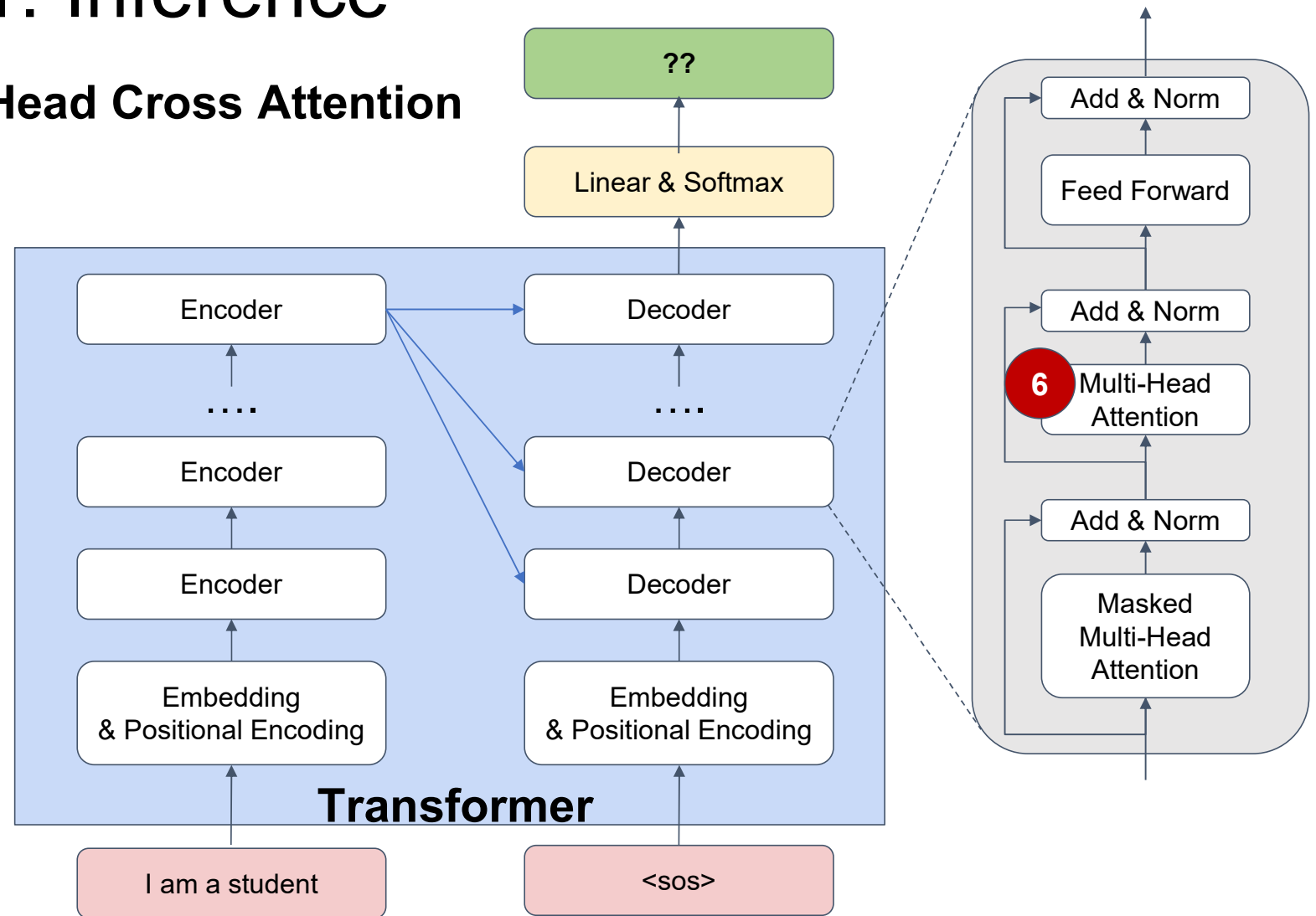
- Decoder takes in entire output sequence (gold answer).
- Mask future tokens to prevent them from affecting current prediction.



⌘ **Inference phase:** Since future tokens do not exist, the mask is not needed.

Transformer: Inference

6. Decoder - Multi-Head Cross Attention



Transformer: Inference

6. Decoder - Multi-Head Cross Attention

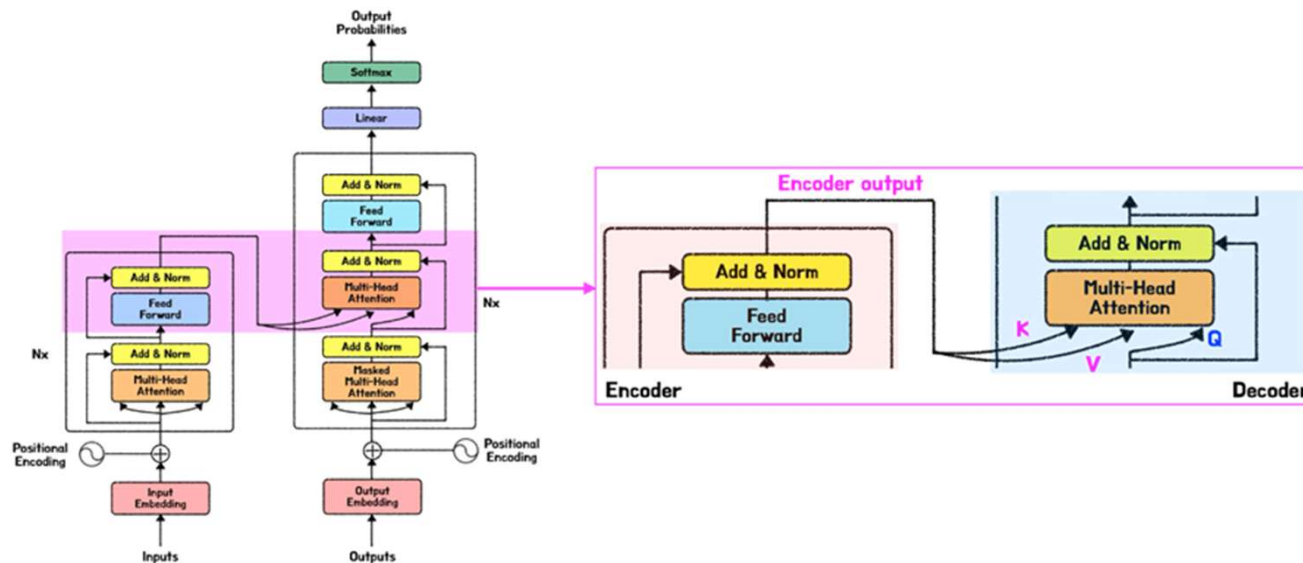
🔗 Calculate relevance of current output tokens to tokens from input sequence.

🔗 Query from Decoder

- Hidden state that captures the context of the output sequence generated up to the current time step.

🔗 Key, Value from Encoder

- Contextualized representation of the entire input sequence.



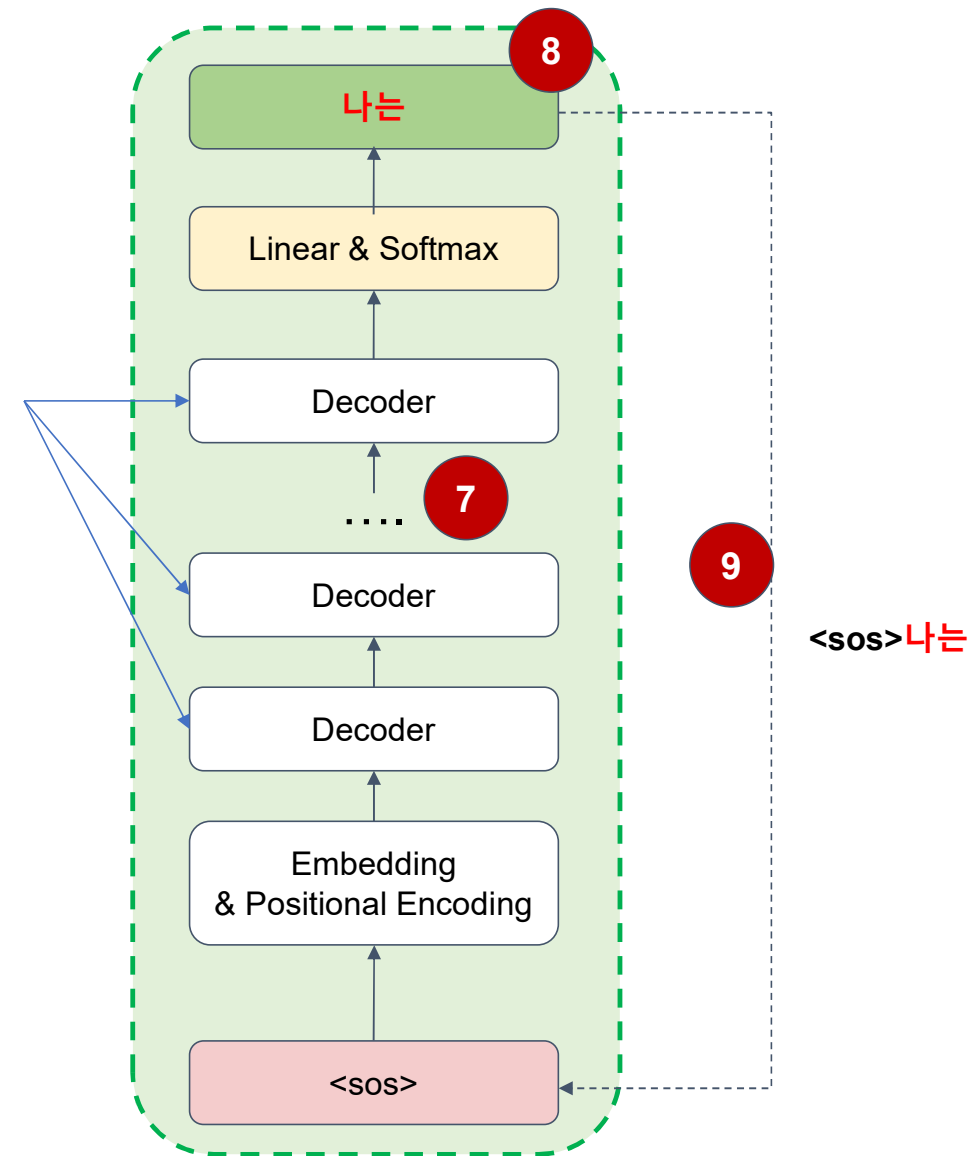
Transformer: Inference

7. Decoder - Repetition across all layers

8. Decoder - Linear & Softmax with Vocabulary

- 1. Final attention output is passed through Linear & Softmax layer.
- 2. Looks at Vocabulary
- 3. Chooses word with highest score as output token

9. Decoder - Repetition until all tokens are generated



Evolution of Transformer Architectures

- 🔗 The original Transformer (**Encoder-Decoder model**) was designed for **sequence-to-sequence tasks (especially translation)**.
 - **Encoder** understands the input by **capturing the full context and transforming it into a semantic representation**.
 - **Decoder** generates the output using the Encoder's representation and previously generated tokens.
- 🔗 **LLM inference is not a seq2seq task, but an autoregressive generation with a single sequence.**
 - **The input already provides sufficient context**, which the model can directly leverage through **Self-Attention**.
 - Is an Encoder really necessary for LLM Inference?
⇒ **Decoder-only model !**

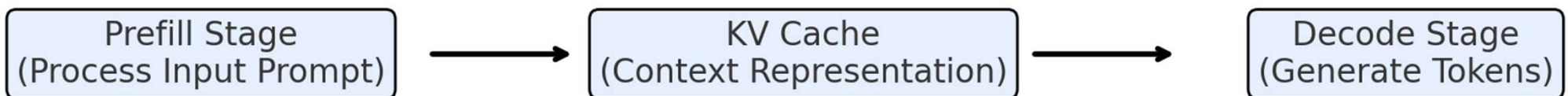
Decoder-only Model

🔗 Decoder-only Model

- Specialized in **autoregressive generation** that predicts the next token sequentially from the input. (**Next-Token Prediction**)
- Used in **GPT** (OpenAI), **LLaMA** (Meta), Gemma (Google)

🔗 LLM Inference Flow

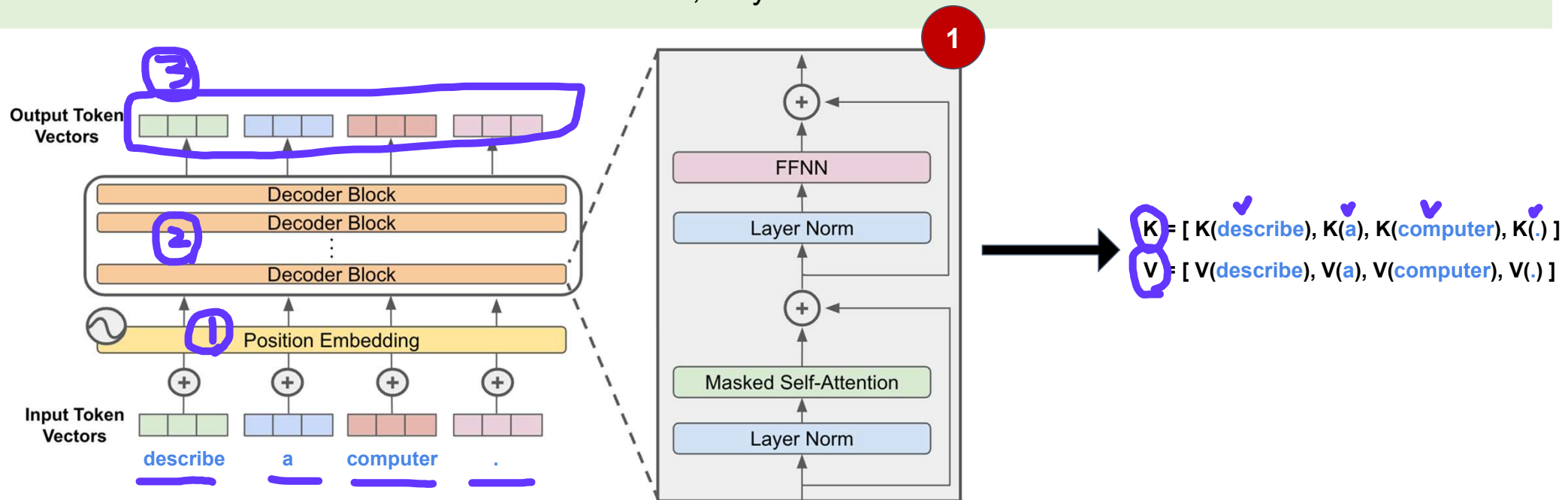
- **Prefill Stage**
 - Processes the entire input (prompt)
 - Computes the **KV cache** to capture context
 - Equivalent to the **encoding role** in Encoder-Decoder model.
- **Decode Stage**
 - Uses stored KV cache to **incrementally generate tokens**
 - Equivalent to the **decoding role** in Encoder-decoder model



Decoder-only Model

LLM Inference Flow

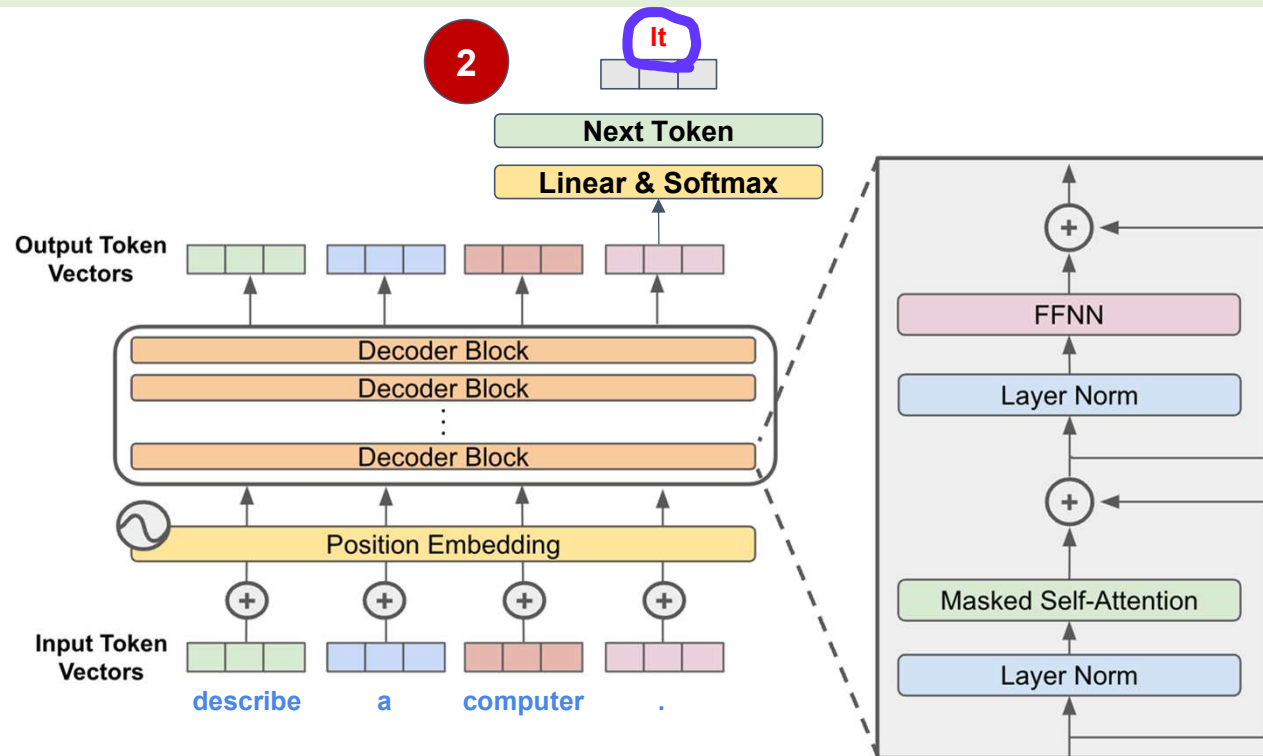
- Prefill Stage
 - 1. **Compute KV Cache** for the entire input prompt
 - **KV Cache**: stored Key/Value vectors from past tokens, reused for generation
 - **No Multi-Head Cross-Attention**, only Masked Self-Attention



Decoder-only Model

LLM Inference Flow

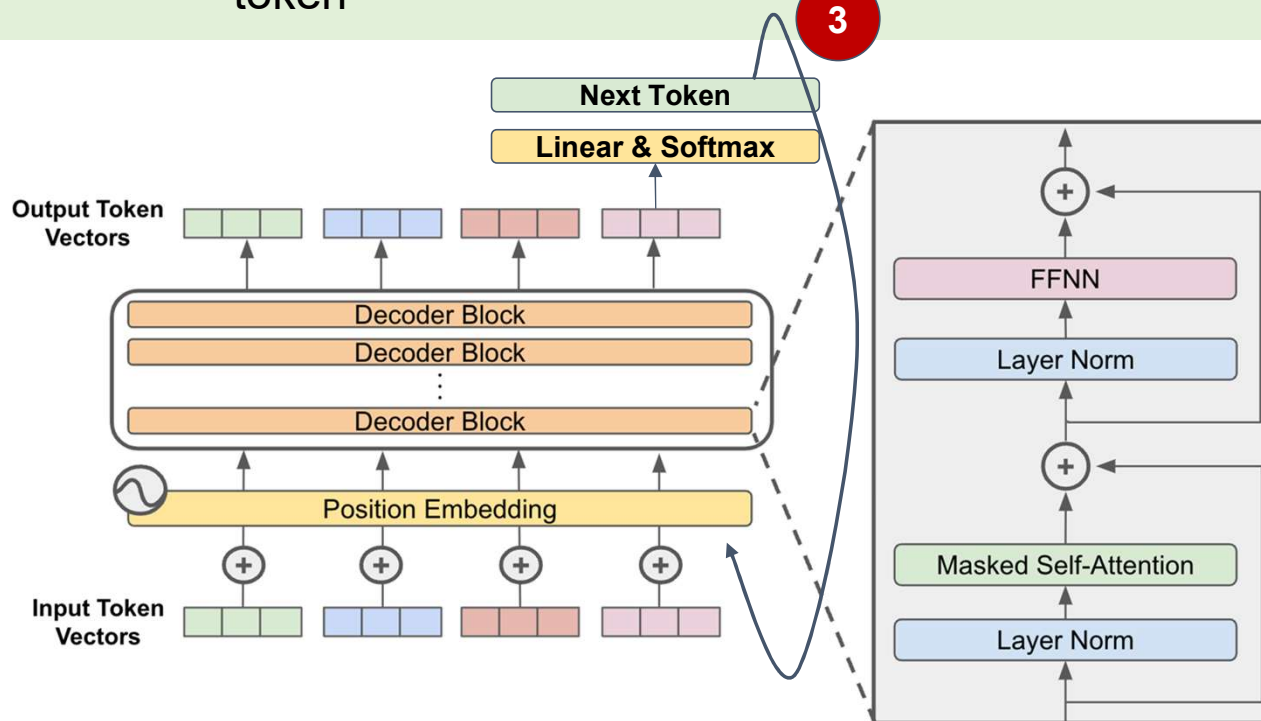
- Prefill Stage
 - 2. Apply Softmax on the last token to **predict the next token** using Vocabulary



Decoder-only Model

LLM Inference Flow

- Decode Stage
 - 3. Append the newly generated token to the KV cache and use it to predict the next token



Step1.

$K = [K(\text{describe}), K(a), K(\text{computer}), K(.), K(\text{It})]$

$V = [V(\text{describe}), V(a), V(\text{computer}), V(.), V(\text{It})]$

Step2.

$K = [K(\text{describe}), K(a), K(\text{computer}), K(.), K(\text{It}), K(\text{is})]$

$V = [V(\text{describe}), V(a), V(\text{computer}), V(.), V(\text{It}), V(\text{is})]$

...

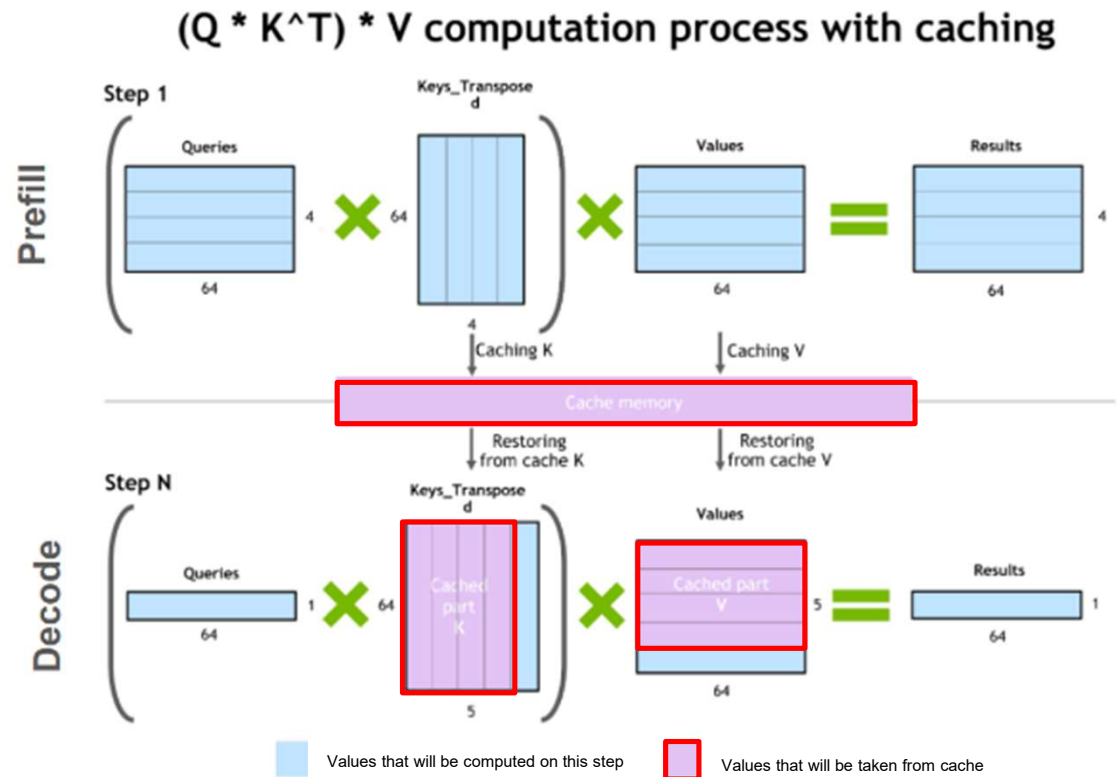
KV Caching

Key-Value Caching

Input이 커질수록 KV 캐시는 커진다.

What is KV Caching?

- ❖ Common optimization for the decode phase
- ❖ The decode phase generates a single token at each time step, but **each token depends on the key (K) and value (V) tensors** of all previous tokens
- ❖ To **avoid recomputing all these (K, V) tensors** for all tokens at each time step, it's possible to **cache them (K, V) in GPU memory**
- ❖ Every iteration, when new elements are computed, they are simply added to the running cache to be used in the next iteration



KV Cache GPU Memory Issue

🔗 Two main contributors to the GPU LLM memory requirement.

- **Model weights:** The model parameters.
 - Ex) Llama-2-7B (FP16) would take roughly $7B * \text{sizeof}(\text{FP16}) \approx 14\text{GB}$ in memory.
- **KV cache:** The caching of self-attention to avoid redundant computation.

🔗 **KV cache:** Memory size is required for each token in the input sequence, across the batch of inputs.

- **Total Size of KV Cache in bytes** = $\text{batch size} * \text{sequence length} * 2 (\text{key, value}) * \text{num layers} * \text{hidden size} * \text{sizeof}(\text{FP16})$
- Ex) Llama-2-7B model in 16-bit precision and a batch size of 16:
 - The Size of KV Cache = $16 * 4096 * 2 * 32 * 4096 * 2 \text{ bytes} \approx \mathbf{32GB}$

GPU	Memory (GB)	Memory bandwidth (GB/s)	FP16 Tensor Core (TFLOP/s)
A10	24	600	125
A100 PCIe/SXM	80	1935/2039	312
L4	24	300	121
L40S	48	864	362
H100 PCIe/SXM	80	2000/3350	756/989
H200 SXM	141	4800	989

We already faces GPU OoM in A10 and L4 GPUs!!!

What if we use larger LLM models like 13B or 70B?



Efficient Memory Management

- Paged Attention in vLLM -



Motivation: Serving LLMs Efficiently

- Goal: maximize the throughput of LLM inference serving
 - LLM servers handle many concurrent requests on expensive GPUs.
 - Higher throughput means more requests per GPU, and lower cost per request.
- The bottleneck is GPU memory
 - Model weights are fixed in size, but the **KV cache grows with every generated token.**
 - How efficiently we manage the KV cache directly limits how many requests can run at once.



Background: The KV Cache

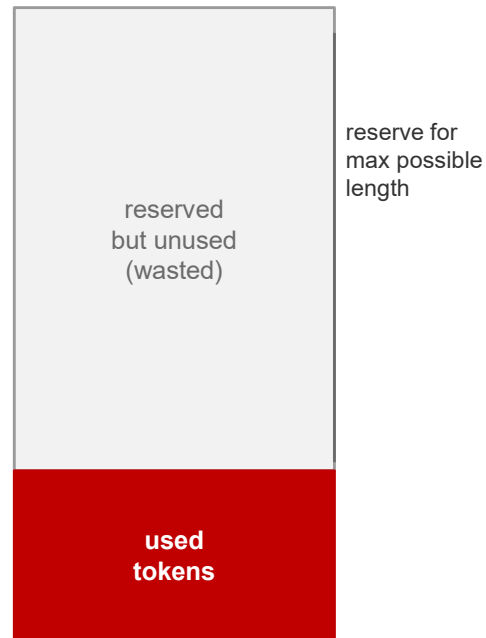
- What is the KV cache?
 - To generate each new token, the model attends to all previous tokens.
 - The Key and Value tensors of past tokens are **cached to avoid recomputation**.
- Why it is hard to manage
 - The KV cache is large, dynamic, and grows one token at a time.
 - Its final size is unknown in advance: it depends on the output length.
 - For large models, the KV cache can exceed the size of the model weights themselves.

The Problem: Memory Fragmentation

- Naive systems store each request's KV cache in one contiguous chunk
 - Space is reserved up front for the maximum possible sequence length.
- Internal fragmentation
 - Reserved-but-unused slots are wasted, because the real output is usually shorter.
- External fragmentation
 - Pre-allocated chunks of different sizes leave unusable gaps between them.
- Result: only about 20–40% of KV cache memory holds useful tokens.

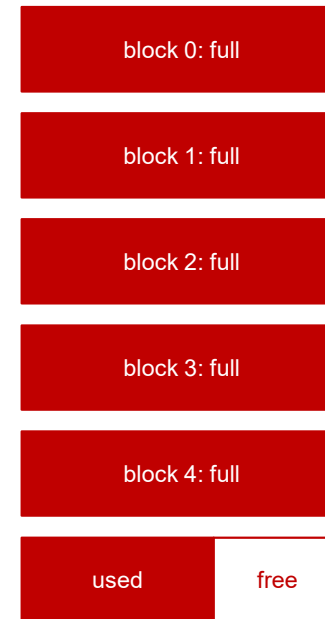
Where Does the Memory Go?

Contiguous Allocation (naive)



Most of the block sits empty

PagedAttention (paged)



Only the last block is partial
→ tokens packed tightly, near-zero waste

Key Idea: Apply OS Paging to the KV Cache

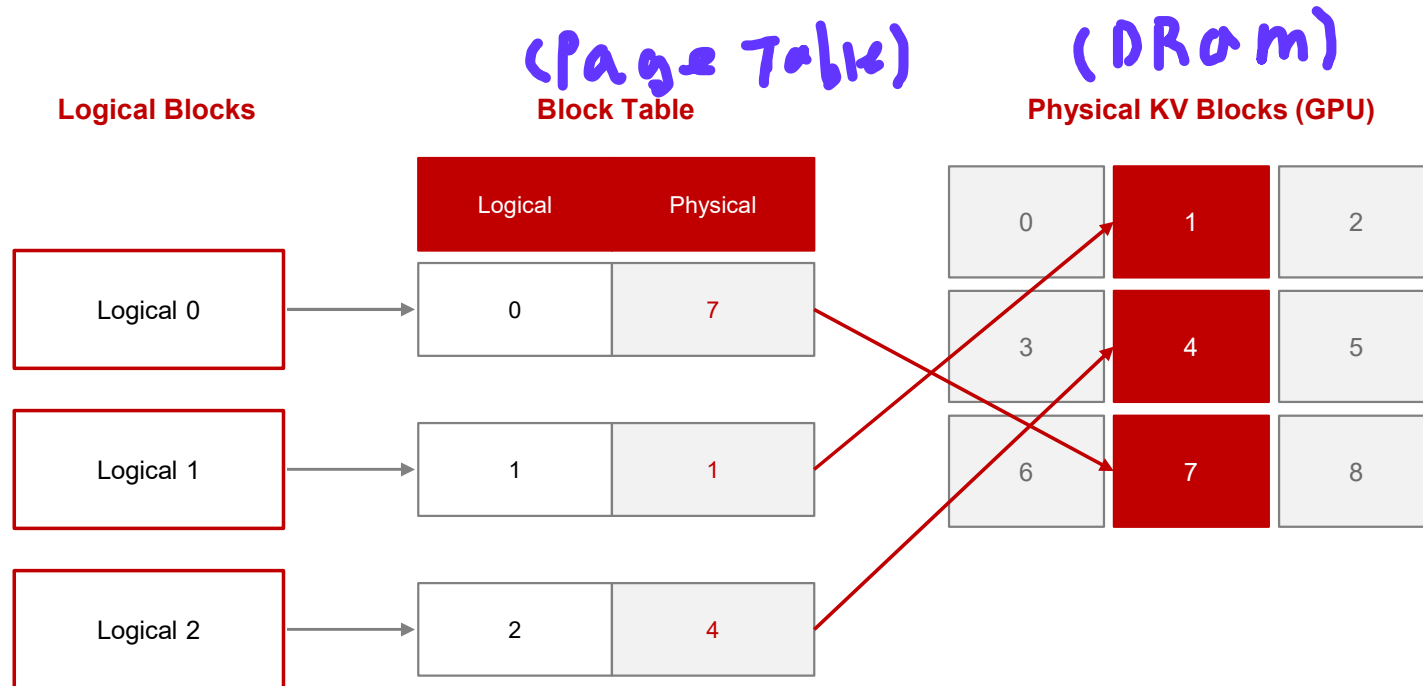
- Insight: treat the KV cache like virtual memory
 - Operating systems solved fragmentation decades ago using **paging**.
 - PagedAttention borrows the same idea for the GPU KV cache.
- Mapping the analogy
 - **Process** → **Request**
 - **Page** → **KV block (a fixed number of tokens)**
 - **Page table** → **Block table**
 - **Scattered physical pages** → **scattered KV blocks in GPU memory**
- Because KV blocks need not be contiguous, fragmentation nearly disappears.



PagedAttention: Core Concepts

- Logical Blocks
 - A request's tokens, grouped into fixed-size blocks (e.g., 16 tokens each), in order.
- Physical Blocks
 - The actual KV data, stored in slots **scattered across GPU DRAM**.
- Block Table
 - Maps each logical block to its physical block, exactly like a page table.
- Filled
 - The number of token slots currently occupied within a block.

Block Table: Logical → Physical Mapping



Each logical block can map to any physical block — they need not be contiguous in GPU memory.



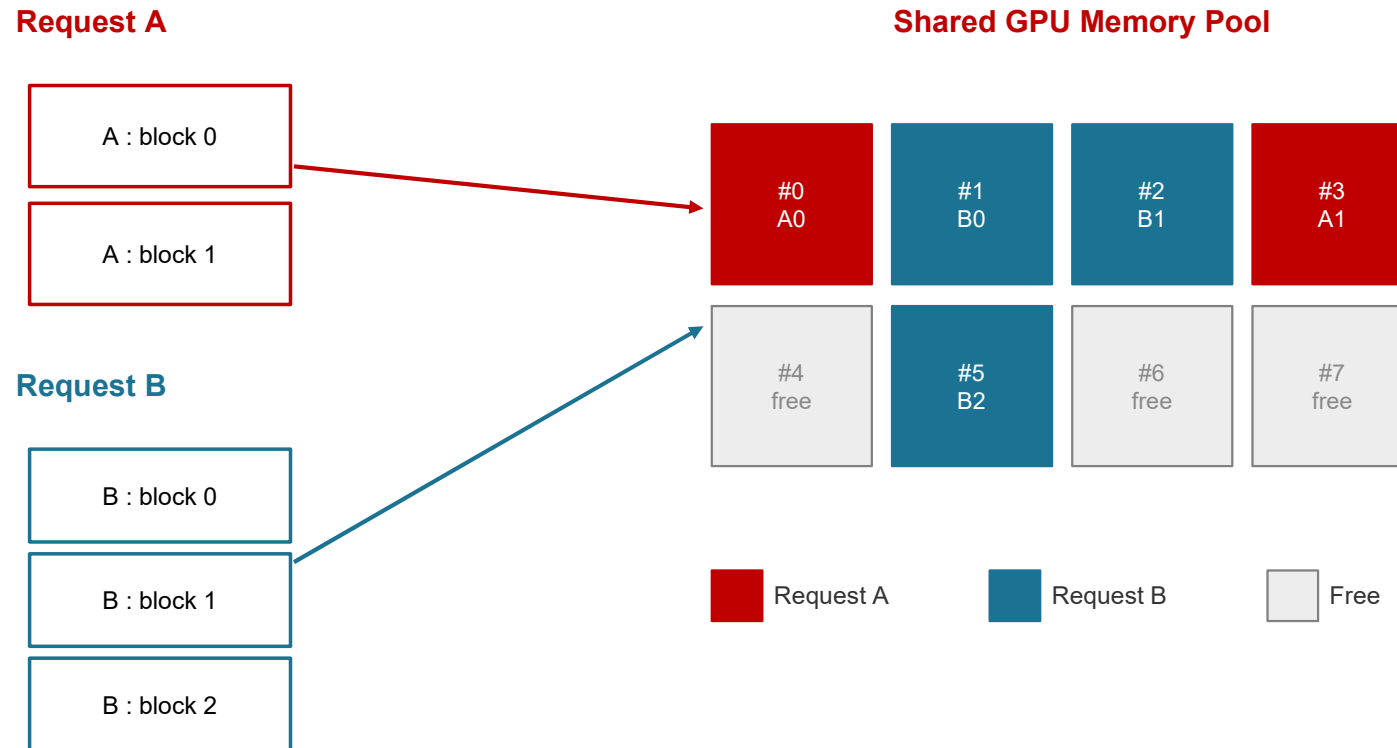
How It Works: Generating Tokens

- On-demand allocation
 - Physical blocks are allocated **dynamically, only as tokens are produced.**
 - Each decode step writes one new KV entry into the next free slot.
- Filling blocks
 - The current block fills slot by slot; when it is full, a new physical block is allocated.
- Near-zero waste
 - Only the **last block** of a sequence can be partially filled.
 - Every other block is fully utilized.



PagedAttention with Multiple Requests

Serving Multiple Requests



Concurrent requests share one physical pool; their blocks interleave freely, with no fragmentation.



Memory Sharing: Copy-on-Write

- Sharing identical blocks
 - Requests with a common prefix (e.g., the same prompt) can **share physical blocks**.
 - Ideal for parallel sampling and beam search, where outputs branch from one shared context.
- Copy-on-Write
 - Shared blocks stay read-only while their contents remain identical.
 - When a sequence diverges, **only that one block is copied**; the rest stay shared.
- Impact: large memory savings and higher throughput for branching workloads.

PagedAttention: Summary

- **Problem** storing each request's KV cache contiguously wastes most GPU memory to fragmentation.
- **Key Idea** apply OS paging — split the KV cache into fixed-size blocks mapped by a block table.
 - Logical blocks (contiguous tokens) map to physical blocks scattered in GPU memory
 - Non-contiguous storage eliminates external fragmentation
- **Benefits**
 - Near-zero waste: only the last block of a sequence is partially filled
 - Dynamic, on-demand block allocation during decoding
 - Block sharing and copy-on-write across concurrent requests
- **Result** far higher serving throughput from the same GPU hardware.



RAG: Retrieval-Augmented Generation

Limitation of LLM-Only Inference

🔗 LLMs are rapidly **scaling up in size**.

- Ex) GPT-1 (117M) → GPT-2 (1.5B) → GPT-3 (175B) → GPT-4 (estimated at 1.8T)

🔗 However, despite their scale, these models are **still constrained by fixed training data and inherent limitations**.

- **Knowledge Cutoff & Hallucination**

- LLMs remain bounded by the **knowledge available at the time of training** and generate answers that **sound correct but are wrong**.
- **Retraining** such large models to incorporate new knowledge is **too expensive**.

⇒ **RAG(Retrieval-Augmented Generation)**: LLMs enhanced with **external knowledge retrieval** without retraining

RAG: Retrieval-Augmented Generation

🔗 Why are RAG-based LLM Services gaining attention?

- Knowledge Cutoff & Hallucination
 - LLM remain bounded by the **Knowledge available at the time of training** and generate answers that **sound correct but are wrong**.

"How many index types does the latest version of Milvus support?"

User Query



LLM
(Large Language Model)

Output

A

Wrong Answer
"As of last month, Milvus support 158 Index types."

<Knowledge Cutoff and Hallucination Problems>

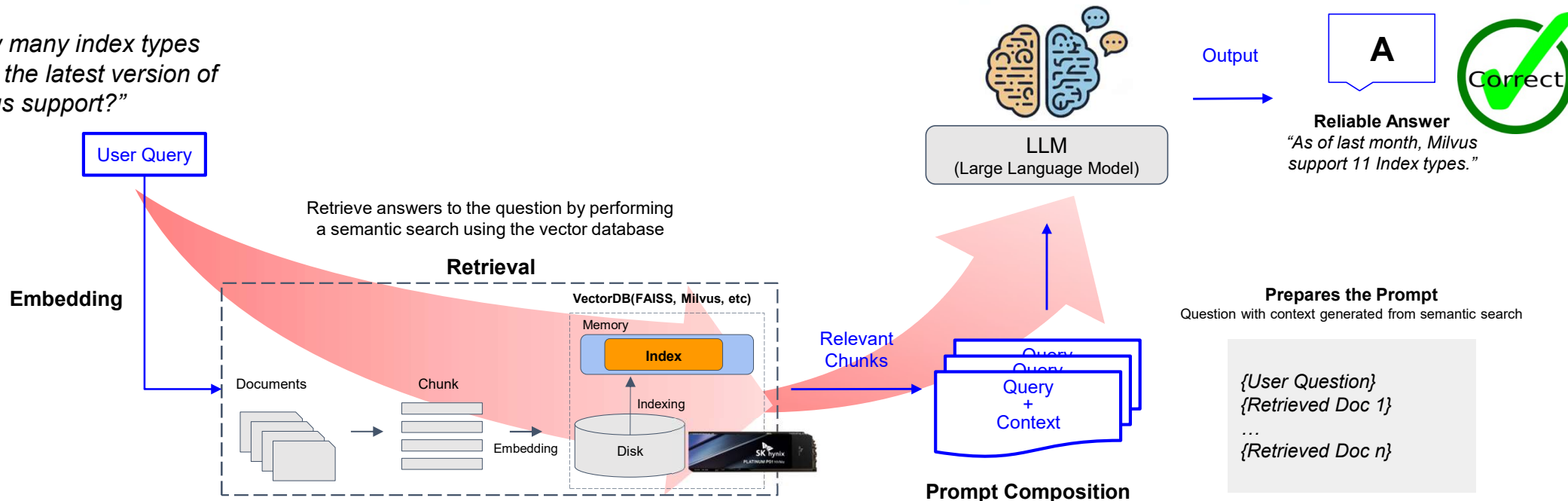


RAG: Retrieval-Augmented Generation

RAG Benefits

- **Fresh Knowledge Access**) Retrieve up-to-date information beyond model's training cutoff
- **Grounded & Accurate Answers**) Reduce hallucinations by citing real documents and sources
- **Domain Adaptability**) Inject domain-specific knowledge without costly fine-tuning
- **Data Privacy & Security**) Use proprietary or sensitive documents at runtime without retaining

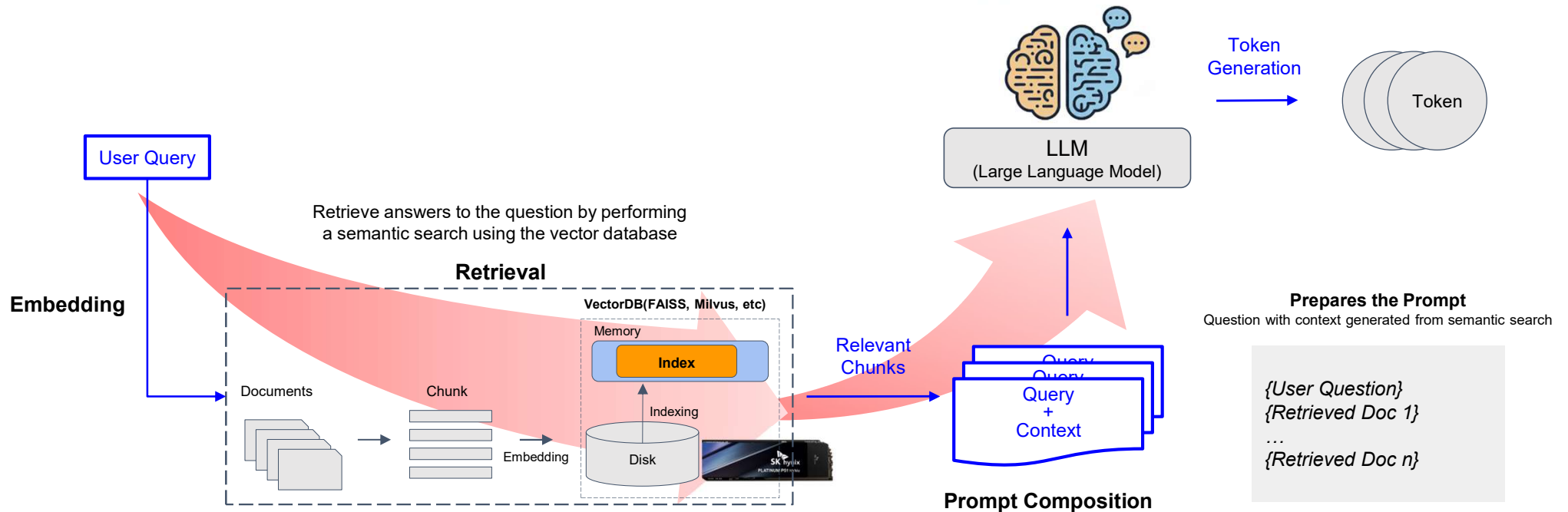
"How many index types does the latest version of Milvus support?"



RAG: Retrieval-Augmented Generation

RAG Flow

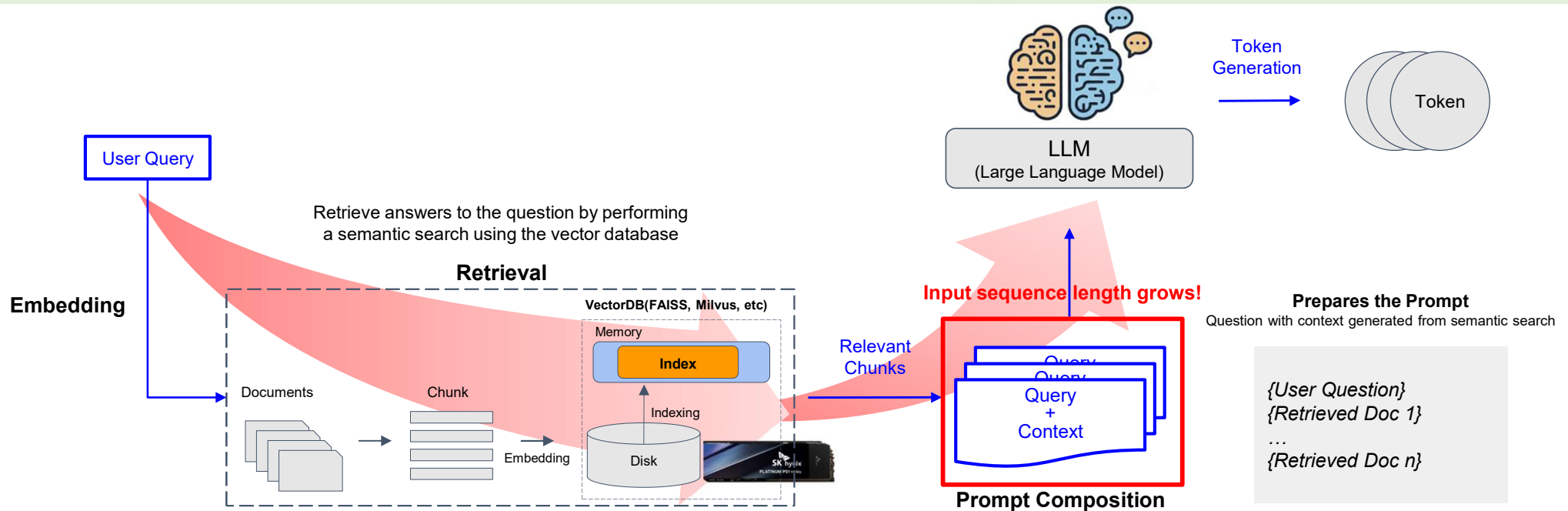
- In **Retrieval** part, documents are chunked and embedded, then stored in a **VectorDB**.
- **Approximate Nearest Neighbor (ANN) search** retrieves the **most relevant chunks**, which are **appended to user query** and passed into LLM.



RAG: Retrieval-Augmented Generation

🔗 **RAG** expands the input sequence, increasing GPU memory pressure.

- Retrieved document chunks are appended to the prompt, causing the context length to grow rapidly.
- (Recap) *Total Size of KV Cache in bytes* = *batch size* * *sequence length* * 2 (*key, value*) * *num layers* * *hidden size* * *sizeof(FP16)*



LMCache

- **Distributed Storage**

- LMCache enables KV cache storage across multiple storage tiers including GPU memory, **CPU DRAM**, and **local disk** to overcome memory limitations.

- **Cross-Instance Reusability**

- Cached data can be shared and reused across all serving instances, supporting flexible caching of any reusable text regardless of prefix patterns.

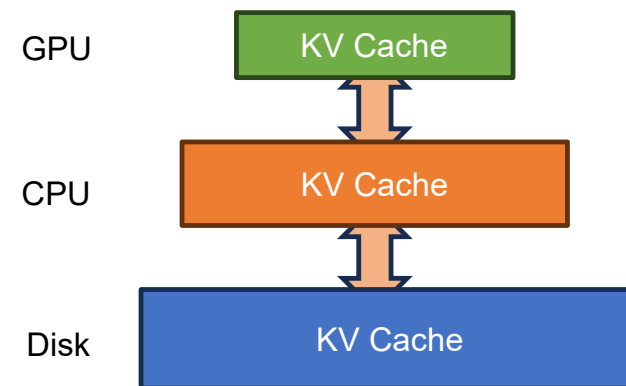


Use case 1: **long context**
Context length: **25K** tokens
(Llama 70B on A40)



Use case 2: **RAG**
Retrieved chunks: **4 x 2K** tokens
(Llama 70B on A40)

LMCache drastically reduces the prefill delay (TTFT) by reusing KV caches



VectorDB Overview

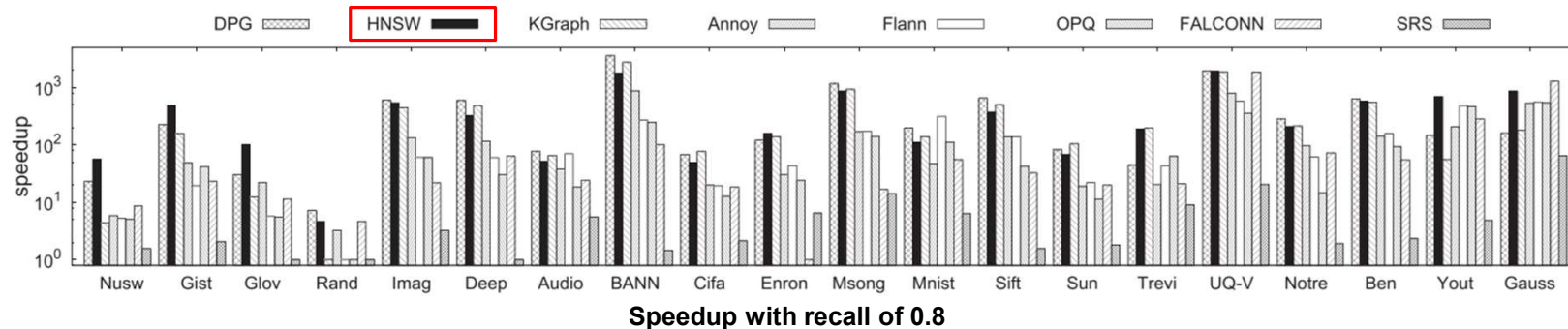
🔗 VectorDB is core knowledge store in RAG.

- Stores the latest documents and data to enable LLMs to **leverage external knowledge**.
- Widely adopted open-source systems include **FAISS(Meta)** and **Milvus(Zilliz)**.

🔗 The foundation is **Approximate Nearest Neighbor (ANN)** indexing.

- **Rapidly compares query vectors** with stored vectors to retrieve the most relevant results.
- Balances trade-off among accuracy, speed, and memory usage.
- Examples: **HNSW** (graph-based), IVF (partition-based), PQ (quantization-based)

HNSW is a fast & accurate across multiple datasets!



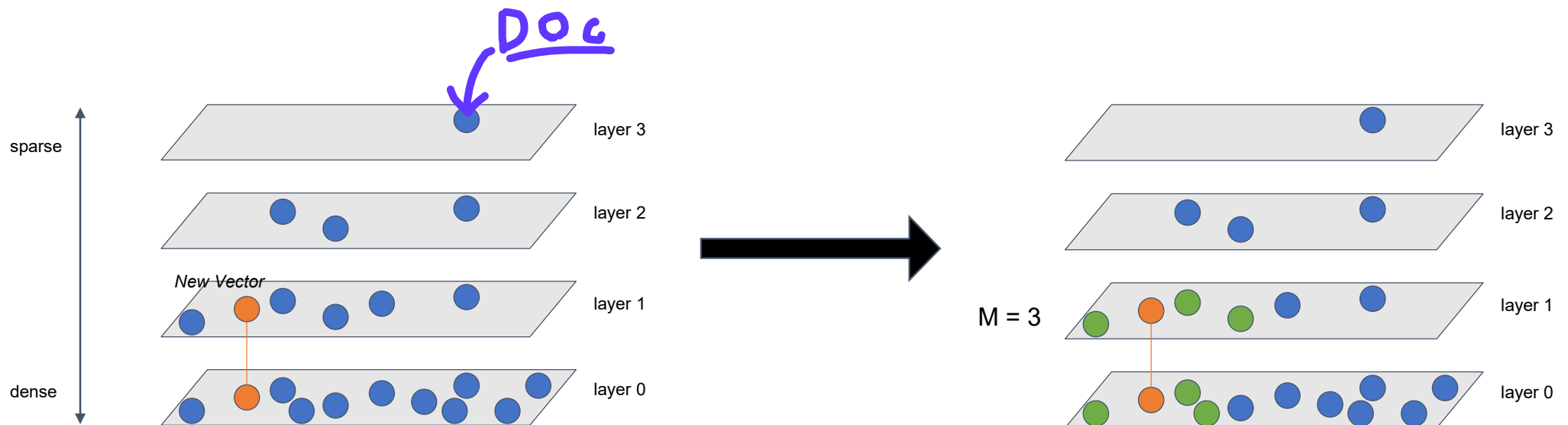
[4] Li, Wen, et al. "Approximate nearest neighbor search on high dimensional data—experiments, analyses, and improvement." *IEEE Transactions on Knowledge and Data Engineering* 32.8 (2019): 1475-1488.

[5] Y. A. Malkov et al, Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs, in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836

HNSW

HNSW Index Building

- HNSW builds its index by inserting new vector into a **randomly assigned maximum layer**, then **connect it to nearest neighbors within each layer** from top to bottom.
 - M is the maximum number of neighbor links each node can have.

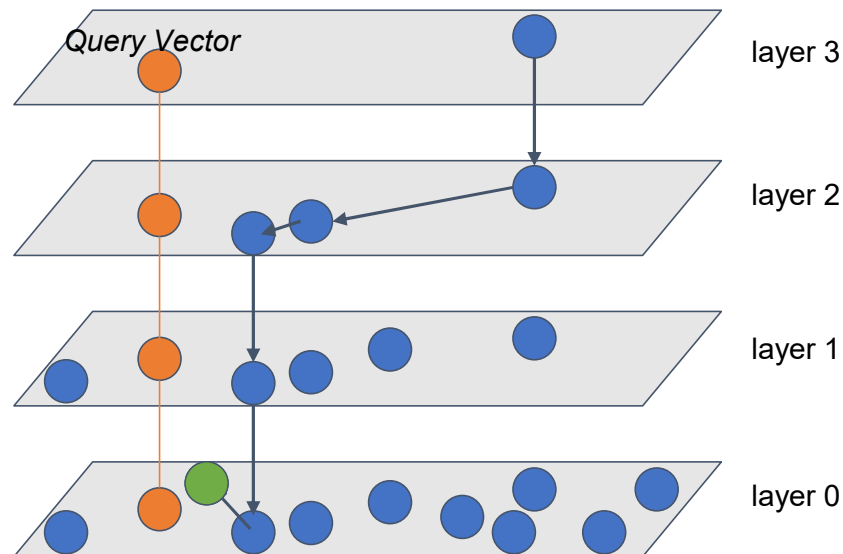


HNSW

지역성 문제를 해결하기 위해 가장 상위 layer부터 찾아감.

HNSW Index Search

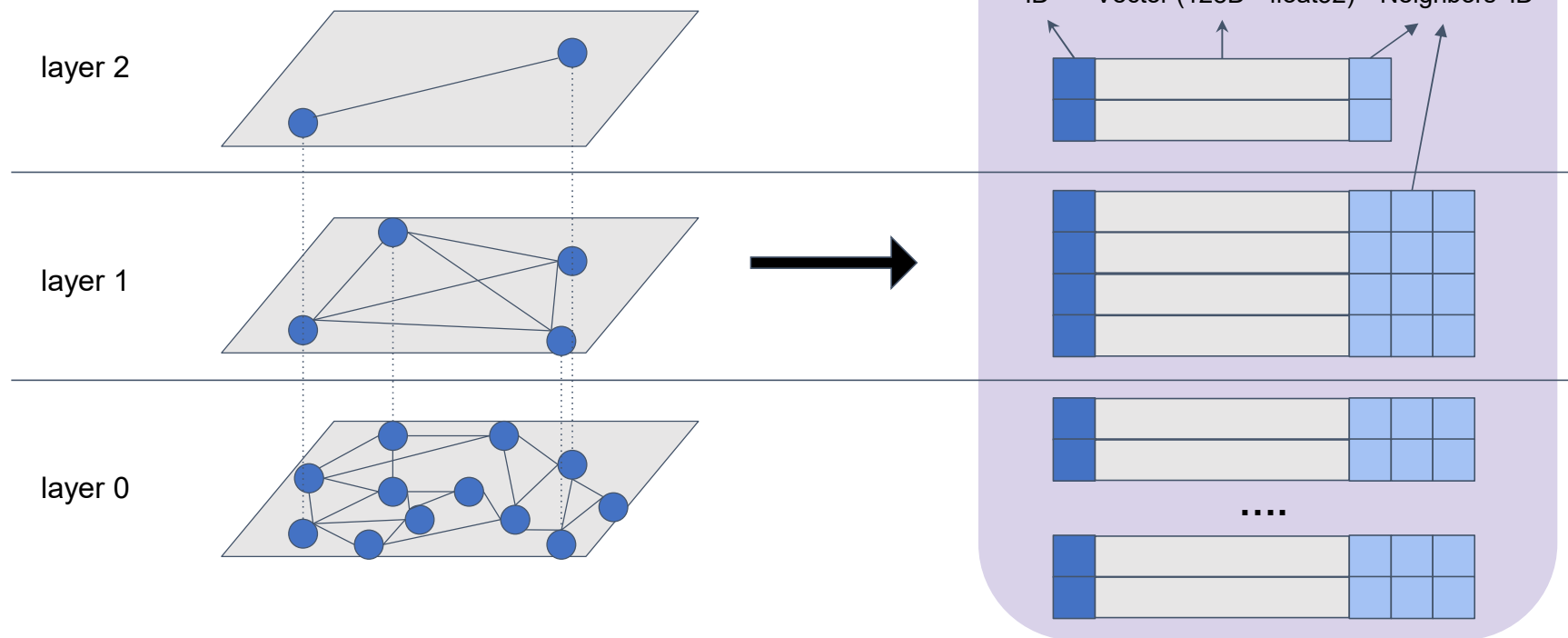
- HNSW search starts from the upper layers and greedily traverses the graph, iteratively moving the query vector toward its nearest neighbors while maintaining a candidate set.



Limitation of HNSW

🔗 HNSW Trade-off

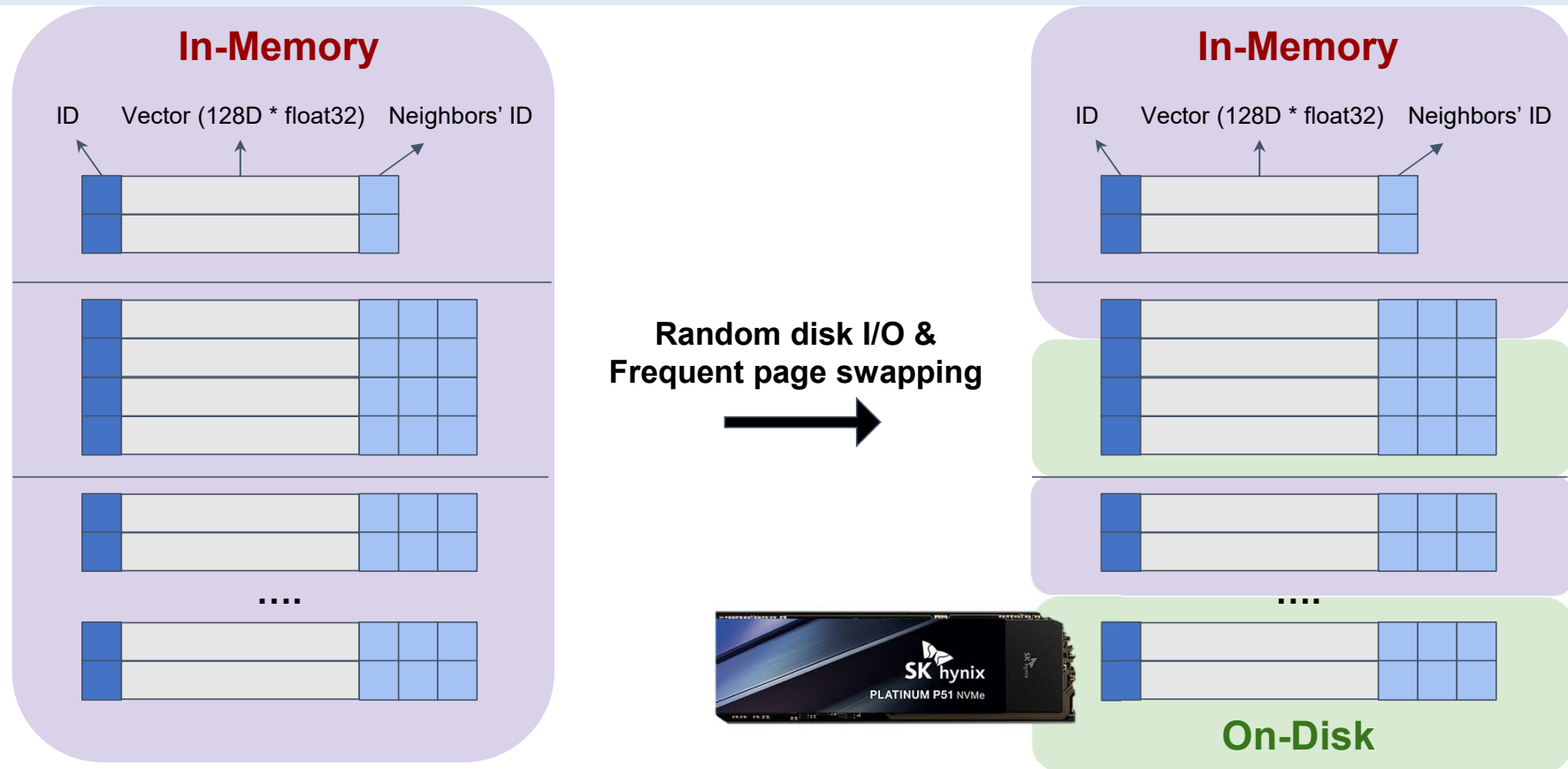
- Advantage: High Recall, Low Latency
- Disadvantage: **High Memory Usage**
 - **Edge, Node replication across levels**



Limitation of HNSW

🔗 HNSW with Partial On-Disk Storage

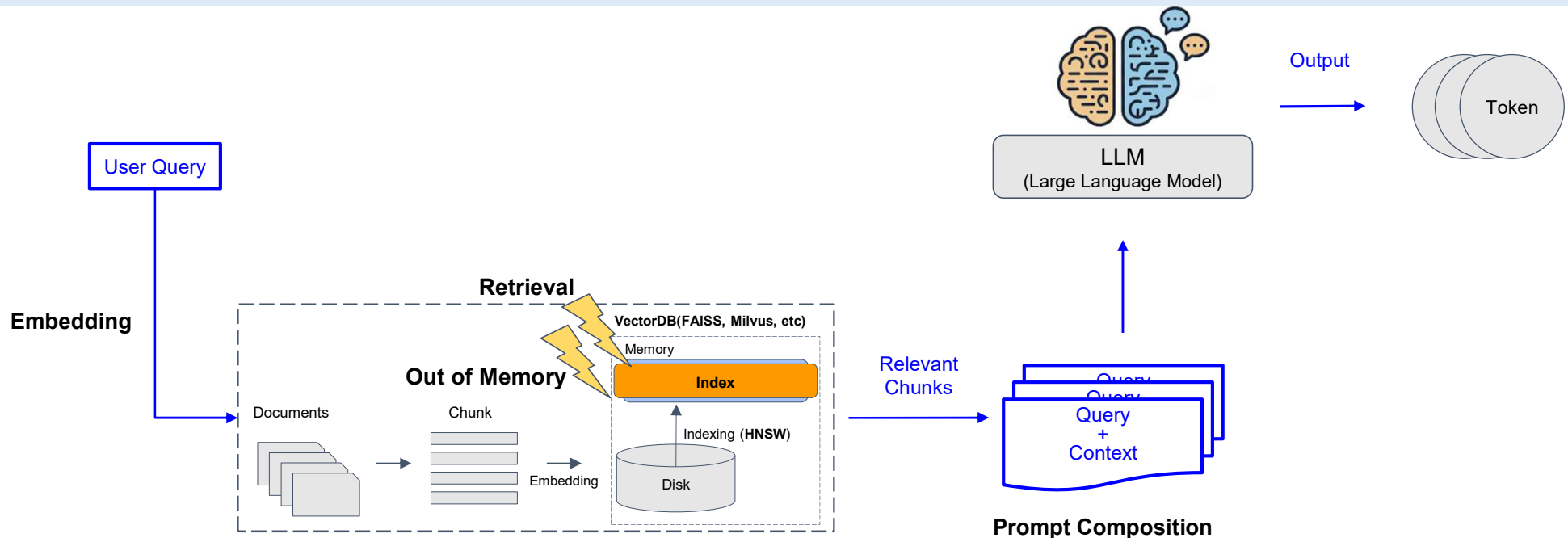
- Random node access(SSD Swap in and out) → Unpredictable latency spikes may occur.



Limitation of HNSW

⌘ Limitation of In-Memory ANN Approaches

- The **memory constraint** has created a need for storage-efficient alternatives to handle datasets **beyond RAM capacity**.
- **OoM**(Out of Memory) can make vector search the bottleneck, increasing **TTFT**(Time To First Token) latency.



Disk-based Approaches

On-Disk Approaches

- HNSW relies heavily on RAM to achieve low latency and high recall
- As data volume grows or embedding dimensionality increases,
 - Cost becomes prohibitively high
 - Scalability significantly degrades
- DiskANN is a graph-based index that significantly reduces memory requirements by storing the index (graph + embedding vectors) on disk

Indexing strategy of DiskANN

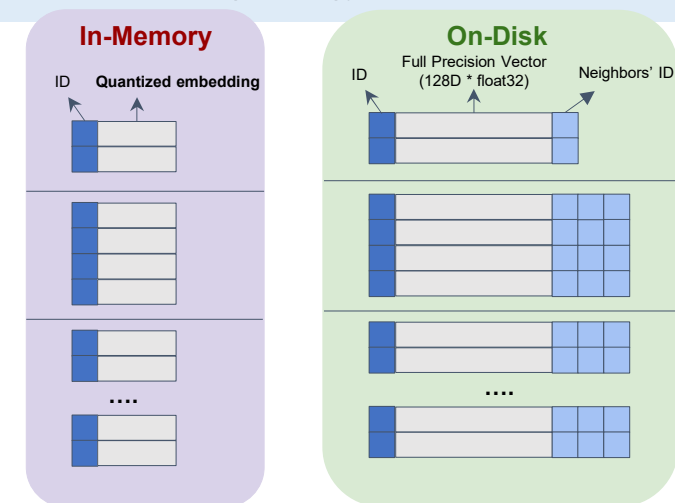


TABLE 3

Parameters for Comparison Between Hierarchical NSW and Faiss on a 200M Subset of 1B SIFT Dataset

Algorithm	Build time	Peak memory (runtime)	Parameters
Hierarchical NSW	5.6 hours	64 Gb	$M = 16, efConstruction = 500$ (1)
Hierarchical NSW	42 minutes	64 Gb	$M = 16, efConstruction = 40$ (2)
Faiss	12 hours	30 Gb	$OPQ64, IMI2x14, PQ64$ (1)
Faiss	11 hours	23.5 Gb	$OPQ32, IMI2x14, PQ32$ (2)

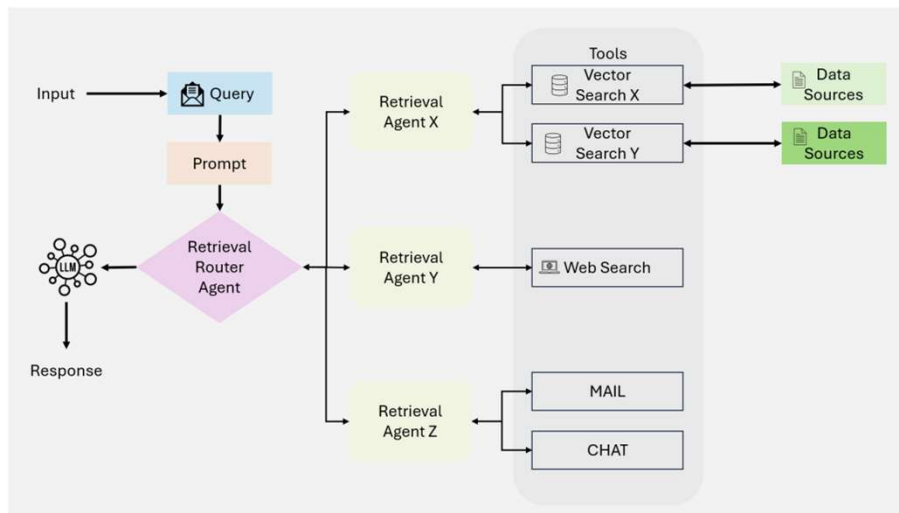
Memory requirements of HNSW

- [5] Y. A. Malkov et al, Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs, in IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 42, no. 4, pp. 824-836
- [6] Jayaram Subramanya, Suhas, et al. "Diskann: Fast accurate billion-point nearest neighbor search on a single node." *Advances in neural information processing Systems* 32 (2019).

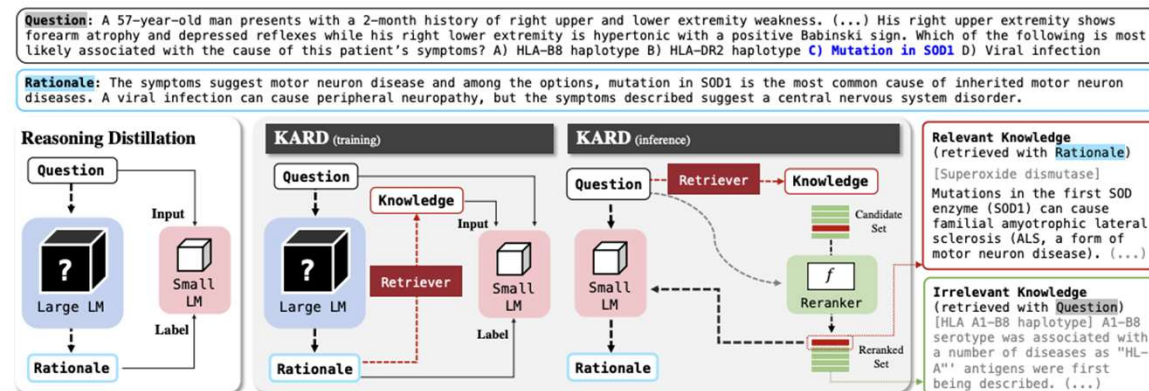
Agentic AI & SLM with Knowledge base

🔗 A reliable knowledge base helps overcome the core limitations of Agentic AI and SLM(Small Language Model).

- **Agentic AI:** Errors from one agent's incorrect output can cascade through step-by-step collaboration **without a trusted knowledge base**.
- **SLM:** Limited knowledge capacity and outdated information.



Combination of Multi-Agent AI and Knowledge Bases



Combination of SLM and Knowledge Bases

[7] Singh, Aditi, et al. "Agentic retrieval-augmented generation: A survey on agentic rag." *arXiv preprint arXiv:2501.09136* (2025)

[8] Kang, Minki, et al. "Knowledge-augmented reasoning distillation for small language models in knowledge-intensive tasks." *Advances in Neural Information Processing Systems* 36 (2023): 48573-48602.